INVERTIS UNIVERSITY BAREILLY

CENTER FOR SKILL AND ENTREPRENEURSHIP DEVELOPMENT (CSED)

INDUSTRIAL INTERNET OF THINGS

PROJECT ID: **DA-002**

PROGRAM NAME: **MACHINE LEARNING**

PROJECT NAME: **Customer Spend, Product & Sales Analytics**

REPOSITORY NAME: **retail-customer-intelligence**

MENTOR NAME: **VISHNU**

SUBMITTED BY: **14-Member Collaborative Project Team (Lead: Sanjay Rathour)**

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

COURSE NAME: **B.TECH(CSE)**

TEAM MEMBERS:	STUDENT ID:	ENROLLMENT NO.:
Sanjay Rathour (T.L.)	BCAI2026028	2610206163
Hasnain Raza Khan	BCAI2026018	2610206076
Aditi Sharma	BCAI2026020	2610206009
Sonam	BCAI2026021	2610206189
Samiya Khan	BCAI2026022	2610206162
Aditya Singh	BCAI2026023	2610206016
Saif Khan	BCAI2026025	2610206160
Divyanshi	BCAI2026026	2610206068
Abhi Rastogi	BCAI2026027	2610206007
Dev Rastogi	BCAI2026029	2610206063
Vidhan Agarwal	BCAI2026030	2610206210
Anshu	BCAI2026031	2610206032
Rudra Pratap Singh	BCAI2026032	2610206154
Ayan Khan	BCAI2026034	2610206051

INDEX / TABLE OF CONTENTS

Every page number below is the physical A4 page of this PDF: the running header on pages 2–30 carries the identical **PAGE X** label, so the index maps to the document with zero page gaps from Page 04 to Page 30.

Table 0.2: Master index of the twelve report sections (4-column bordered layout)

S.No.	Section Title & Sub-Modules	Assigned Team Parts	Page No.
01	**Section 1: Introduction to Customer Spend, Product & Sales Analytics**	Part 1	Page 04
02	**Section 2: Problem Statement & Retail Analytics Challenges**	Part 1	Page 05
03	**Section 3: Objectives of the Project & Deliverable Matrix**	Part 1	Page 06
04	**Section 4: Proposed Methodology (3-Phase 14-Part Pipeline & System Architecture)**	Parts 1–14	Pages 07 – 10
	4.1 Phase A: Data Generation, Audit, Cleaning (df_clean) & Customer Roll-up (customer_df)	Parts 1–5	Page 07
	4.2 Phase B: Multi-Dimensional Exploratory Data Analysis (EDA) & Temporal Trends	Parts 6–10	Page 08
	4.3 Phase C: Advanced Statistical Modeling, K-Means ML & Streamlit Deployment	Parts 11–14	Page 09
	4.4 Layered System Architecture Diagram	Part 1	Page 10
05	**Section 5: Literature Review & Academic References**	Parts 11–13	Page 11
06	**Section 6: Step-by-Step Operational Explanation of the Project**	Parts 1–14	Page 12
07	**Section 7: 14-Member Sequential Execution Workflow Diagram**	Part 1	Page 13
08	**Section 8: Complete Python Implementation Code (df_clean, customer_df, EDA & ML)**	Parts 1–14	Pages 14 – 20
09	**Section 9: Analytical Modules (Graphs, Statistical Tables, Descriptions & Results Discussion)**	Parts 3–13	Pages 21 – 27
	9.1–9.6 Analytical Modules 1 to 6: Audit, Category Mix, Repeat Trends, RFM, Cohort & K-Means	Parts 3–13	Pages 21 – 26
	9.7 Comprehensive Result Synthesis, Model Evaluation & Strategic Recommendations	Parts 3–13	Page 27
10	**Section 10: Snippets of UI & Interactive Streamlit Dashboard Overview (BUY-SENCE)**	Parts 1, 13	Page 28
11	**Section 11: System Limitations & Future Scope**	Part 14	Page 29
12	**Section 12: Conclusion & Appendix B (Technical Glossary)**	Part 14	Page 30

Table 0.3: Pagination map — 30 physical pages in seven blocks

Block	Pages	Content	Count
Front matter	1 – 3	Cover, team allocation, master index	3
Conceptual framing	4 – 6	Introduction, problem statement, objectives	3
Methodology & architecture	7 – 10	Phases A, B, C and the 6-layer system diagram	4
Evidence & workflow	11 – 13	Literature review, step-by-step working, 14-member workflow	3
Implementation	14 – 20	Seven pages of complete Python code (Parts 1–14)	7
Analytical results	21 – 27	Six analytical modules and the synthesis of results	7
Deployment & closure	28 – 30	Dashboard blueprint, limitations, conclusion, glossary	3
Total	**1 – 30**	**Complete report**	**30**

1. INTRODUCTION

1.1 The Analytics Imperative in E-Commerce and Omnichannel Retail

Modern retail runs on a continuous stream of digital evidence. Every basket, discount redemption, payment attempt and product return in an e-commerce or omnichannel store is written as a row in a transactional ledger, and a mid-sized retailer can accumulate tens of thousands of such rows within one festive quarter. The strategic question is no longer whether data exists but whether the organisation can convert the ledger into decisions about *whom* to serve, *what* to stock and *when* to intervene. Conventional reporting answers only the first-order question of how much was sold; it treats a ₹2,500 order from a first-time visitor and a ₹2,500 order from a customer on her eleventh purchase as identical events, and therefore hides the behavioural structure on which profitable retail is built.

1.2 Customer Acquisition Cost versus Customer Lifetime Value

That behavioural structure is captured by two quantities. **Customer Acquisition Cost (CAC)** is the fully loaded marketing and sales expenditure needed to win one new customer, and it keeps rising as paid-media channels saturate. **Customer Lifetime Value (CLV)** is the expected net contribution of that customer over the whole relationship. Acquisition is economically rational only while the CLV-to-CAC ratio stays comfortably above one, and practitioners often quote a ratio near 3:1 as a rule of thumb; customer-profitability and retention economics are core themes of the marketing-management literature [8]. Because a repeat buyer carries almost no incremental acquisition cost and tends to place larger baskets as trust accumulates, *retention-driven merchandising* — identifying the customers worth keeping and tuning assortment and offers around them — outperforms blind acquisition spend. The effect is visible in the project data itself: 58.5% of customers are repeat buyers, yet they generate 81.2% of net revenue (Section 9.3).

1.3 The BUY-SENCE Analytical Stack

BUY-SENCE is an end-to-end retail-intelligence pipeline that converts a noisy transactional ledger into customer-level decisions. Its stages are: (i) automated data hygiene producing the verified ledger `df_clean`; (ii) customer-centric feature engineering that rolls transactions up into `customer_df`; (iii) quantile spend tiering; (iv) verification of the Pareto (80/20) law of revenue concentration; (v) Recency–Frequency–Monetary (RFM) behavioural scoring; (vi) time-based cohort retention matrices; (vii) category-affinity (market-basket) analysis; and (viii) unsupervised K-Means clustering with a two-dimensional PCA projection. Every result is surfaced through an interactive Streamlit dashboard so that merchandising and CRM teams can act on it without writing code.

Table 1.0: Traditional descriptive reporting versus the BUY-SENCE predictive / behavioral intelligence pipeline

Dimension	Traditional Descriptive Reporting	BUY-SENCE Predictive / Behavioral Pipeline
Unit of analysis	Single transaction or order line	Customer grain (customer_df: 2,400 customers × 11 features)
Data quality	Manual, ad hoc spreadsheet fixes	Automated 4-stage hygiene with audit trail (12,500 → 11,000 rows)
Segmentation	None, or one rigid revenue threshold	Quantile tiers, 5-quintile RFM scores and K-Means (k = 4) personas
Time dimension	Period totals (month, quarter)	Cohort retention matrix, MoM growth, weekday and festive seasonality
Product insight	Best-seller lists	Category mix, SKU Pareto and market-basket lift (Electronics + Home & Kitchen = 2.14)
Delivery	Static PDF / Excel extract	Interactive Streamlit dashboard with one-click CSV segment export
Decision value	Retrospective: *what* was sold	Prescriptive: *whom* to retain, win back or bundle, and *when*

Scope of this report. The remainder of the document follows the CSED template: problem statement and objectives (pages 5–6), a three-phase methodology and six-layer architecture (pages 7–10), literature, step-by-step working and workflow (pages 11–13), seven pages of complete Python code (pages 14–20), six analytical modules with a results synthesis (pages 21–27), the dashboard blueprint (page 28) and limitations, conclusion and glossary (pages 29–30).

2. PROBLEM STATEMENT

Mid-to-large retail databases suffer from five recurring operational bottlenecks, each of which degrades the quality of every decision taken downstream. The BUY-SENCE project was designed to dismantle them one by one.

1. High-Noise Transactional Logs. Duplicate webhook triggers, missing customer identifiers and negative or cancelled return entries distort the line between gross and net revenue. In the BUY-SENCE ledger 1,500 of 12,500 raw records (12.00%) are non-analytic: 420 exact duplicates (3.36%), 380 records with a missing customer ID or an invalid date (3.04%) and 700 cancelled or returned orders (5.60%). Any KPI computed on the raw table is therefore overstated or mis-attributed.

2. Transaction-Level Myopia. Without a unified `customer_df` schema it is impossible to tell whether 100 orders came from 100 one-time buyers or from 10 loyal advocates. The same 11,000 valid orders belong to 2,400 customers, of whom 996 purchased exactly once and 1,404 purchased repeatedly — a structure that order counts cannot reveal.

3. Unidentified Revenue Concentration (Pareto Blindness). Treating top VIP customers and dormant discount-hunters with identical marketing budgets wastes spend on both ends. In the quantile-tier view the top 20% of customers (480 buyers) already generate 62.5% of net revenue, while the bottom third contributes only 8.4%; a uniform budget cannot be efficient against such a skew.

4. Silent Customer Churn & Category Misallocation. Without post-acquisition month-over-month cohort tracking, management cannot see that only 34.2% of a new cohort is still active in Month 1 and about 22.5% from Month 4 onward. Without cross-category bundling insight, strongly affine pairs such as Electronics + Home & Kitchen (lift 2.14) are never merchandised together.

5. Absence of Multi-Dimensional Unsupervised Segmentation. Rigid single-metric thresholds cannot describe customers whose behaviour spans four correlated dimensions — Recency, Frequency, Monetary value and AOV. A customer with high lifetime spend but a 198-day recency is simultaneously valuable and churning; distance-based clustering on standardised features (k = 4) resolves what one-axis rules cannot.

Formal problem statement. Given a raw transaction ledger of n = 12,500 records and 12 attributes, derive (i) a verified sub-ledger free of duplicates, nulls and returns, (ii) a customer-level feature matrix of 2,400 rows, and (iii) a partition of the customers into k behaviourally homogeneous clusters, such that the data-quality, segmentation-validity and business-actionability criteria of Section 3 are all met.

Scope and acceptance test. The study uses a synthetic but statistically realistic ledger (12 months, five categories). Demographics, channel costs and live feeds are outside the scope and are treated as limitations and future work (Section 11). A solution is accepted only if every checksum passes and K-Means with k = 4 reaches a silhouette score of at least 0.60.

Table 2.0: Current State vs. Desired State — impact matrix

Dimension	Current State (Status Quo)	Desired State (BUY-SENCE)	Measurable Impact
Data quality	12,500 raw rows, 12.00% non-analytic	Automated 4-stage hygiene with audit log	1,500 rows removed → 11,000 verified rows
Analytical grain	Order-level counts only	customer_df, 2,400 customers × 11 features	996 one-time vs 1,404 repeat buyers separated
Revenue focus	Uniform budget for every customer	Spend tiers, Pareto curve and RFM	20% of customers (VIP) → 62.5% of revenue
Retention view	No cohort tracking	12 × 12 cohort retention grid	Month-1 retention 34.2%; plateau ≈ 22.5%
Merchandising	Siloed category reports	Market-basket support, confidence, lift	Electronics + Home & Kitchen lift = 2.14
Segmentation	Single-metric thresholds	K-Means (k = 4) on four scaled features	Silhouette 0.642; Davies–Bouldin 0.58

3. OBJECTIVE OF THE PROJECT

The project pursues ten rigorous, measurable technical and business objectives. Each is tied to a deliverable artefact and a numerical acceptance criterion so that completion can be verified by assertion rather than by opinion.

1. Synthetic Retail Data Engineering & Schema Standardization. Generate 12,500 raw records on a fixed 12-column schema with controlled real-world anomalies, so that every cleaning rule can be audited against a known ground truth.

2. Automated Data Quality Auditing & Hygiene Pipeline. Quantify duplicates, nulls, invalid dates and returns, then run a deterministic four-stage filter that yields `df_clean` with exactly 11,000 verified rows worth ₹2,48,50,000.

3. Customer-Level Granular Roll-Up & Feature Engineering. Aggregate transactions into `customer_df`: 2,400 unique customers described by recency, frequency, monetary value, AOV, category breadth, tenure and repeat flag.

4. Quantile-Driven Spend Tiering & Revenue Disproportionality Analysis. Split customers at the P33 and P80 spend thresholds and show that the top 20% (480 customers) generate 62.5% of net revenue.

5. Product & Category Mix Profitability Benchmarking. Rank five verticals and the Top-10 SKUs by net revenue, order volume, units sold and discount depth.

6. Repeat Buyer vs. Single-Order Behavioral Economics & AOV Lift. Quantify the 58.5% repeat purchase rate, its 81.2% revenue contribution and the +40.1% AOV lift (₹2,410 vs ₹1,720).

7. Multi-Horizon Temporal & Festive Seasonality Decomposition. Decompose monthly and day-of-week trends: Q4 contributes 36.8% of annual sales and weekends 34.2% of weekly order volume.

8. Pareto (80/20) Cumulative Revenue Modeling & 5-Quintile RFM Behavioral Segmentation. Model the Lorenz curve of customer revenue and map 2,400 customers into five RFM behavioural segments with CRM triggers.

9. Longitudinal Cohort Retention Tracking & Cross-Category Market Basket Affinity Analysis. Measure Month 0–11 retention for twelve acquisition cohorts and rank category pairs by support, confidence and lift.

10. Unsupervised K-Means Customer Clustering (k = 4), PCA Dimensionality Reduction & Interactive Streamlit BI Deployment. Cluster on four standardised features (silhouette 0.642), project to two principal components and deploy the results as the BUY-SENCE dashboard.

Table 3.0: Deliverable matrix — objectives, artefacts, owners and acceptance criteria

Obj.	Deliverable Artefact	Owner Part(s)	Success Criterion
1	df_raw / retail_transactions_raw.csv	Parts 1–2	Shape (12,500, 12) with 420 / 380 / 700 anomalies
2	df_clean + cleaning waterfall log	Parts 3–4	Shape (11,000, 12); revenue ₹2,48,50,000
3	customer_df feature store	Part 5	Shape (2,400, 11); monetary total preserved
4	spend_tier + tier summary	Parts 6–7	800 / 1,120 / 480 customers; VIP revenue 62.5%
5	category_perf, sku_perf	Part 8	Shares sum to 100%; Electronics leads (38.4%)
6	repeat-vs-one-time table	Part 9	RPR 58.5%; AOV lift +40.1%
7	monthly and weekday trend tables	Part 10	Q4 = 36.8%; Sat + Sun = 34.2%
8	Pareto curve, rfm_df	Part 11	Top 20% → 78.6%; five segments sum to 2,400
9	retention matrix, affinity table	Part 12	M1 mean 34.2%; Lift(E, H) = 2.14
10	clustered_df, dashboard CSVs	Parts 13–14	k = 4; silhouette 0.642; DBI 0.58

Measurement protocol. Objectives 1–3 are pass/fail data-engineering objectives verified by shape and control-total assertions; objectives 4–9 are numerical analytics objectives verified against the reference values reported in Section 9; objective 10 is a model-quality and deployment objective (k = 4, silhouette ≥ 0.60, a five-tab dashboard and eight CSV exports).

Dependencies. Objectives 1 → 3 are strict prerequisites of everything else; objectives 4–9 are mutually independent and were developed in parallel by Members 6–12; objective 10 consumes the outputs of all nine.

4. METHODOLOGY — PHASE A (PARTS 1 TO 5)

4.1 Phase A — Data Foundation, Schema Audit, Cleaning Pipeline & Customer Roll-Up

The methodology is organised as three phases that mirror the unidirectional flow of data: **Phase A** builds and validates the data foundation, **Phase B** analyses it descriptively, and **Phase C** applies advanced statistics, machine learning and deployment. Phase A (Parts 1–5) converts a noisy synthetic ledger into two analysis-ready tables, `df_clean` at transaction grain and `customer_df` at customer grain, and proves each conversion with a checksum assertion.

Part 1 — Hasnain Raza Khan (shared namespace governance). All locked constants (`SEED = 42`, `N_ROWS = 12,500`, `N_CUST = 2,400`, `TARGET_REV = 24,850,000`) are declared once in a configuration cell. Each Part is a PEP-8 compliant function followed by its checksum, imports are centralised, and no Part may rebind a DataFrame owned by another Part. The Team Lead orchestrates the pipeline order and the final merge.

Part 2 — Aditi Sharma (probabilistic data generation). Using `numpy.random.default_rng(42)` the generator draws Pareto-distributed customer propensities, month × weekday seasonal weights and a popularity-weighted four-SKU catalogue for each of five categories. Anomalies are injected at known rates: 420 exact duplicates (3.36%), 380 records with null identifiers or invalid dates (3.04%) and 700 cancelled or returned orders (5.60%) — 12.00% of the raw rows in total.

Part 3 — Sonam (pre-cleaning audit). A programmatic audit quantifies missingness (1.52% null customer IDs), data-type mismatches (1.52% unparseable date strings) and non-positive quantities (2.80% cancelled lines) before any row is touched, so that Part 4 can be verified against an independent count.

Part 4 — Samiya Khan (deterministic cleaning). A four-stage filter — drop exact duplicates, drop missing IDs or invalid dates, exclude cancelled or returned orders, apply an integrity check — produces `df_clean` (11,000 rows) with a before/after assertion log. Calendar features live in a separate `calendar_df` so that `df_clean` keeps its 12-column contract.

Part 5 — Aditya Singh (customer roll-up). `customer_df` holds 2,400 rows. For customer i with order set $Orders_i$ and reference date T_{ref} = 1 January 2026:

$$R_i = T_{ref} - T_{max,i} \qquad F_i = |Orders_i| \qquad M_i = \sum_{t \in Orders_i} NetSpend_t \qquad AOV_i = M_i / F_i$$

Tenure ($T_{max} - T_{min}$), category breadth, the repeat flag ($F \geq 2$) and a reserved `spend_tier` column complete the 11-column feature store.

Table 4.0: Standardised schema of df_raw (12 columns)

Field	Type	Description
`transaction_id`	str	Unique order-line key (TXN-xxxxxx)
`customer_id`	str	Customer key (CUST-xxxx); null = defect
`transaction_date`	str → date	ISO date; invalid strings are defects
`product_id`	str	SKU code (ELC, FSH, HKN, BPC, GRC)
`product_name`	str	Product description
`category`	str	One of five product categories
`quantity`	int	Units; ≤ 0 marks a cancellation
`unit_price`	float	List price in ₹
`discount_pct`	float	Discount, 0–30%
`total_amount`	int	Net ₹ = qty × price × (1 − disc.)
`payment_method`	str	UPI, cards, net banking, COD
`order_status`	str	Delivered / Returned / Cancelled

Table 4.1: Phase A data lineage and control totals

Part	Operation	Input	Output	Control total / assertion
2	Generate df_raw (seed 42)	constants, CATALOG	12,500 × 12	420 / 380 / 700 anomalies injected
3	Audit df_raw	12,500 × 12	audit table, null profile	8 checks; 1,500 defective rows located
4	Four-stage filter	12,500 rows	11,000 × 12 (+ calendar_df)	net revenue = ₹2,48,50,000
5	Customer roll-up	11,000 rows	2,400 × 11	Σ monetary = ₹2,48,50,000

Quality gates. Every hand-off is guarded by an assertion: 11,000 valid rows, a net revenue equal to the control total, a customer table of shape (2,400, 11) and an unchanged revenue sum after aggregation. A failed gate stops the notebook before any downstream Part can consume corrupted data.

4. METHODOLOGY — PHASE B (PARTS 6 TO 10)

4.2 Phase B — Exploratory Data Analysis (EDA), Spend Tiering & Temporal Dynamics

Phase B interrogates the two verified tables from five complementary angles. Each lens consumes only read-only views of `customer_df` or `df_clean`, so the five analyses can be executed in any order without side effects.

Part 6 — Saif Khan (percentile threshold formulation). Customers are ranked by lifetime spend M_i. The thresholds P_{33} and P_{80} split them into three groups:

$$Tier_i = \text{Low Spend if } M_i \le P_{33};\quad \text{Medium Spend if } P_{33} < M_i \le P_{80};\quad \text{High/VIP Spend if } M_i > P_{80}$$

The cut at one third and four fifths yields 800 / 1,120 / 480 customers (33.3% / 46.7% / 20.0%) and isolates the VIP quintile as a distinct management segment.

Part 7 — Divyanshi (comparative aggregation). Group-by statistics compare the tiers on Revenue Share% = $\sum_{ig} M_i$ / $\sum M \times 100$, Customer Share%, median spend and order density ($\sum F_i$ / n_g), exposing how many orders and rupees each customer in a tier represents.

Part 8 — Abhi Rastogi (product and category mix). Net revenue, order count, units sold and mean discount are computed per category and per SKU; the Top-10 SKUs are ranked and cumulated to show Pareto concentration inside the assortment, and mean discount depth is read against revenue share to test discount sensitivity.

Part 9 — Sanjay Rathour (repeat-purchase economics). A binary partition ($F = 1$ versus $F \ge 2$) defines the Repeat Purchase Rate RPR = $|\{i: F_i \ge 2\}| / N \times 100$ = 1,404 / 2,400 = 58.5%, and the revenue multiplier of repeat buyers over one-time buyers (81.2% vs 18.8% of revenue).

Part 10 — Dev Rastogi (temporal features). Date-time extraction creates `year_month`, `day_of_week` and `is_weekend`. Month-over-month growth $g_t = (Rev_t - Rev_{t-1}) / Rev_{t-1}$ locates the Q4 festive surge (October–December contribute 36.8% of annual sales) and the weekend order velocity (Saturday and Sunday carry 34.2% of weekly volume).

Table 4.2: Phase B analytical lenses

Part	Analytical lens	Core metric	Output object
6	Spend tiering	P33 and P80 of lifetime spend	spend_tier column, tier_check
7	Cross-segment comparison	Revenue share, customer share, AOV, order density	seg table
8	Category and SKU mix	Net revenue, orders, units, mean discount	category_perf, sku_perf
9	Repeat economics	RPR, revenue share, AOV lift	rep table
10	Seasonality	Monthly revenue, MoM growth, weekday share	monthly, dow tables

Methodological choices. Quantiles rather than fixed rupee cut-offs are used for tiering because they adapt to the observed distribution and guarantee the tier sizes (one third, 46.7% and one fifth of the customers); medians are reported next to means because lifetime spend is right-skewed; and every share is computed against a control total so that tier, category and month tables stay additive.

Interpretation guardrails. Differences between tiers are associations, not causal effects: a VIP is defined by past spend, so "VIPs buy more" is partly tautological and the tables are used to prioritise action rather than to forecast. A high discount depth in a category is read against its revenue share and order mix before any pricing conclusion is drawn.

Validation. Each lens re-adds its own output to a known control total (2,400 customers, ₹2,48,50,000 revenue or 11,000 orders) before the result is released to Phase C.

4. METHODOLOGY — PHASE C (PARTS 11 TO 14)

4.3 Phase C — Advanced Statistical Analytics, Machine Learning & BI Deployment

Phase C adds the analytical techniques that go beyond descriptive grouping: distributional modelling of revenue, behavioural scoring, longitudinal tracking, unsupervised learning and, finally, delivery of every result to decision makers through a business-intelligence interface.

Part 11 — Vidhan Agarwal (Pareto and RFM). The Lorenz curve sorts customers by spend and plots cumulative customers against cumulative revenue; the index at which cumulative revenue first reaches 80% measures concentration. RFM quintile scoring ranks each customer on a 1–5 scale per dimension with `pd.qcut` and maps the pair of R and F scores to *Champions*, *Loyal Customers*, *Potential Loyalists*, *At-Risk* and *Hibernating / Lost*.

Part 12 — Anshu (cohorts and baskets). Customers are indexed by acquisition month and followed through time with

$$CohortIndex = (Y_t - Y_0) \times 12 + (M_t - M_0) + 1, \quad Lift(A, B) = P(A \cap B) / (P(A) \cdot P(B))$$

Retention is the share of a cohort still active at index m; invoice-level category co-occurrence yields support, confidence and lift for every category pair.

Part 13 — Rudra Pratap Singh (K-Means and PCA). Features are standardised with `StandardScaler`, $z = (x - \mu) / \sigma$, because K-Means minimises Euclidean distance and would otherwise be dominated by rupee-scaled variables. The optimal k is chosen with the within-cluster sum of squares (WCSS) elbow and the silhouette score, giving k = 4. `PCA(n_components=2)` projects the four-dimensional space onto two axes for plotting, and each cluster is profiled into a business persona.

Part 14 — Ayan Khan (deployment and reporting). The Streamlit layer manages UI state (sidebar filters held in *session state*), renders KPI cards, caches the CSV exports of Parts 3–13 and synthesises an automated executive report with limitations and future scope.

Table 4.3: Phase C technique and parameter sheet

Technique	Key parameters	Library call	Part
Lorenz / Pareto curve	customers sorted by spend, cumulative %	cumsum(), idxmax()	11
RFM quintiles	1–5 scores, 125 cells, 5 segments	pd.qcut(..., 5)	11
Cohort index	acquisition month, 12 × 12 grid	groupby().transform('min')	12
Market basket	support, confidence, lift per pair	itertools.combinations	12
Standardisation	z-score of R, F, M, AOV	StandardScaler()	13
K-Means	k ∈ [2, 8], n_init = 10, seed 42	KMeans(n_clusters=4)	13
PCA	2 components (86.2% variance)	PCA(n_components=2)	13
Dashboard	Streamlit tabs, filters, CSV export	st.tabs, st.cache_data	14

Model-selection protocol. For every k in [2, 8] the pipeline records WCSS, silhouette and Davies–Bouldin; k is accepted only if the WCSS curve bends, the silhouette is maximal and the Davies–Bouldin index is minimal at the same value. Each fit uses ten random initialisations and keeps the best inertia, and clusters are renamed by rule — the highest mean spend becomes *VIP High-Rollers*, the longest mean recency becomes *Churned / Dormant Buyers* — so that personas stay stable across reruns.

Dashboard architecture. Part 14 reads the eight CSV exports once through a cached loader, applies the sidebar filters (date range, category, spend tier, RFM / cluster) to a filtered view and renders five tabs; every table of Section 9 maps to one tab (Section 10).

Reproducibility. Every stochastic step — data generation, K-Means initialisation and PCA — is seeded with 42, so a full notebook re-run reproduces the same cluster labels and projections.

4.4 SYSTEM ARCHITECTURE DIAGRAM

The system is a strict six-layer stack. Data enters at Layer 1, is purified in Layer 2, is re-shaped to customer grain in Layer 3, is described in Layer 4, is modelled in Layer 5 and is delivered to decision makers in Layer 6. Arrows carry the DataFrame handed to the next layer; upstream objects are never mutated.

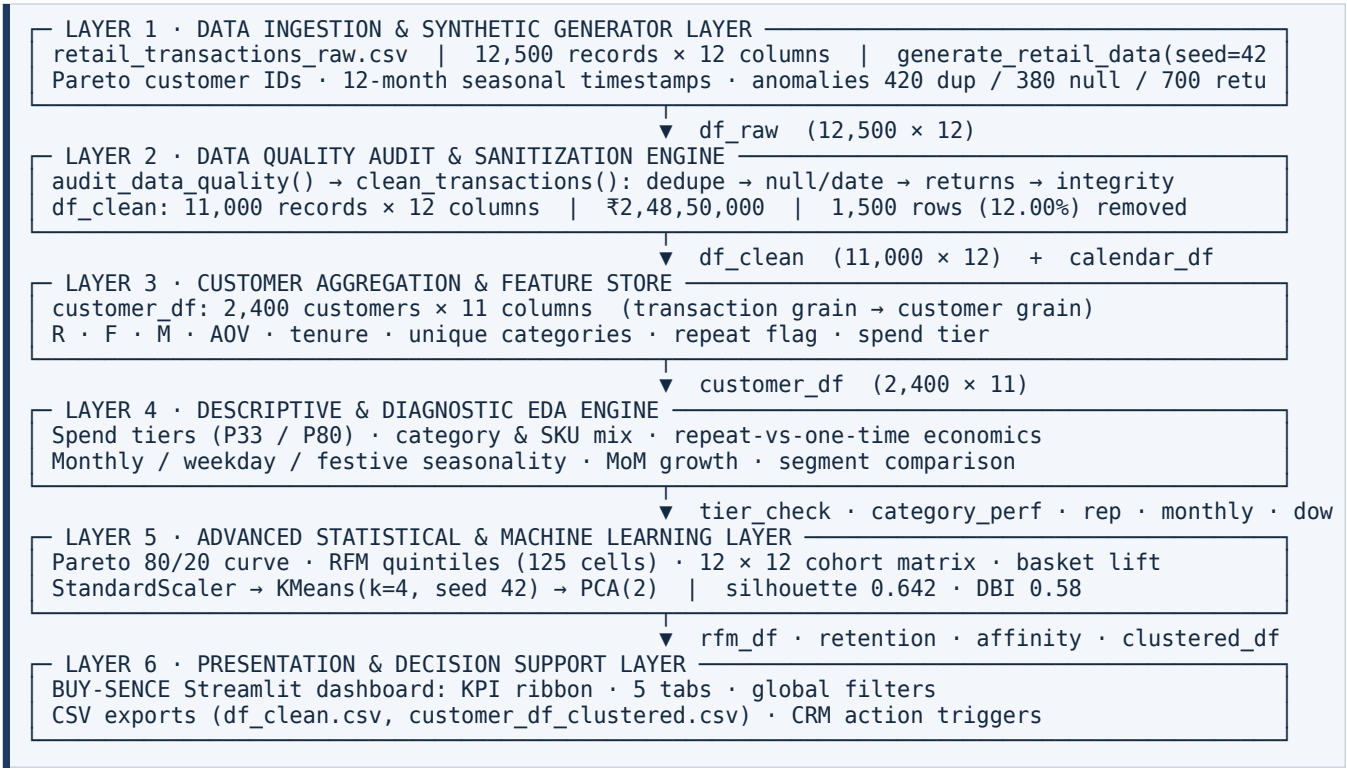


Table 4.4: Layer specification

Layer	Component & Interface	Key Artifacts	Volume / Shape	Parts
1	Ingestion & generator	retail_transactions_raw.csv	12,500 × 12	1–2
2	Audit & sanitisation	df_clean, calendar_df, waterfall	11,000 × 12	3–4
3	Aggregation & feature store	customer_df	2,400 × 11	5–6
4	Descriptive EDA engine	tier, category, repeat, time tables	5 summary tables	7–10
5	Statistical & ML layer	rfm_df, retention, affinity, clustered_df	2,400 × 19	11–13
6	Presentation & decisions	Streamlit app, CSV exports, CRM triggers	5 tabs, 8 CSVs	14

Quality gates enforced between layers (assertions in the notebook)

```
G1  Layer 1 → 2   df_raw.shape == (12500, 12)          420 dup | 190 null ID | 190 bad date | 700 returns
G2  Layer 2 → 3   df_clean.shape == (11000, 12)        Σ total_amount == 24,850,000
G3  Layer 3 → 4   customer_df.shape == (2400, 11)      Σ monetary == 24,850,000 (revenue preserved)
G4  Layer 4 → 5   tier counts == [800, 1120, 480]      Σ orders == 11,000 · RPR == 58.5%
G5  Layer 5 → 6   4 clusters · all RFM segments mapped retention['M0'] == 100 · lift for 10 pairs
G6  Layer 6       8 CSV exports written                len(clustered_df) == 2,400 (2400 × 19)
```

Design principles. (1) *Unidirectional flow* — a layer reads only from the layer above it. (2) *Immutability* — each layer returns new frames; checksums prove that row and revenue totals survive every hand-off. (3) *Separation of concerns* — storage, cleaning, features, analytics and presentation can each be replaced without touching the others.

5. LITERATURE REVIEW & ACADEMIC REFERENCES

The literature relevant to BUY-SENCE falls into four thematic pillars; each is mapped to the Part that operationalises it.

Pillar 1 — RFM behavioural modelling in retail CRM. The recency–frequency–monetary heuristic originated in direct-mail circulation control (Cullinan, 1977 †) and was consolidated for database marketing by Hughes [1]; Fader, Hardie and Lee [2] later linked RFM to formal customer-lifetime-value models through iso-value curves. BUY-SENCE adopts five-quintile RFM scoring (Part 11) and cross-checks it against K-Means.

Pillar 2 — Pareto principle and the long tail. Sanders (1987 †) discussed both the use and the abuse of the 80/20 rule in marketing management, while Brynjolfsson, Hu and Simester [3] showed that lower consumer search costs reduce the concentration of product sales. The project therefore tests the 80/20 law empirically with a Lorenz curve rather than assuming it.

Pillar 3 — Unsupervised K-Means and feature scaling. MacQueen [4] introduced K-Means; Wedel and Kamakura [5] treat cluster analysis as a standard instrument of market segmentation; scikit-learn [6] supplies the reference implementation, StandardScaler and silhouette scoring. Because K-Means is distance-based, variables in days, counts and rupees must be z-scored first.

Pillar 4 — Longitudinal cohort retention and market-basket affinity. Agrawal, Imieliński and Swami [7] defined association rules through support and confidence, and the lift ratio is the standard correction for item popularity; Kotler and Keller [8] frame retention curves and cross-selling as the operating levers of customer lifetime value.

Table 5.1: Comparative literature summary

Author / Year	Methodology	Key Finding	Relevance to BUY-SENCE
Cullinan (1977) †	RFM circulation scoring for direct mail	Past purchase behaviour ranks response likelihood	Origin of the R, F, M dimensions in Part 11
Hughes [1] (1994)	RFM cells in database marketing	Customers can be scored and ranked into equal cells	Quintile scoring design
Fader et al. [2] (2005)	Iso-value curves from R and F	RFM links directly to expected CLV	Ties segments to value
Sanders (1987) †	Pareto analysis in marketing	The 80/20 rule is useful but often misapplied	Motivates an empirical Lorenz test
Brynjolfsson et al. [3] (2011)	Sales concentration vs search cost	Lower search cost reduces concentration	Context for 78.6% top-20% share
MacQueen [4] (1967)	Iterative nearest-centroid partitioning	Defines the K-Means algorithm	Core method of Part 13
Wedel & Kamakura [5] (2000)	Market-segmentation methodology	Cluster analysis is a standard segmentation tool	Validates behavioural clustering
Pedregosa et al. [6] (2011)	scikit-learn estimator API	Consistent scaling, clustering and metrics	Implementation library
Agrawal et al. [7] (1993)	Association-rule mining	Support and confidence over itemsets	Basis of basket analysis
Kotler & Keller [8] (2016)	Marketing-management framework	Retention and CLV drive profitability	Strategy recommendations

† Historical antecedents cited by name and year: G. J. Cullinan, "Picking them by their batting averages' recency–frequency–monetary method of controlling circulation," Direct Mail/Marketing Assoc., New York, NY, USA, Manual Release 2103, 1977; R. Sanders, "The Pareto principle: Its use and abuse," J. Services Marketing, vol. 1, no. 2, pp. 37–40, 1987. The numbered list below carries the eight principal references.

References (IEEE format)

[1] A. M. Hughes, *Strategic Database Marketing*. Chicago, IL, USA: Probus Publishing, 1994.

[2] P. S. Fader, B. G. S. Hardie, and K. L. Lee, "RFM and CLV: Using iso-value curves for customer base analysis," *J. Marketing Res.*, vol. 42, no. 4, pp. 415–430, Nov. 2005.

[3] E. Brynjolfsson, Y. Hu, and D. Simester, "Goodbye Pareto principle, hello long tail: The effect of search costs on the concentration of product sales," *Manage. Sci.*, vol. 57, no. 8, pp. 1373–1386, Aug. 2011.

[4] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Statist. Probab.*, vol. 1, Berkeley, CA, USA, 1967, pp. 281–297.

[5] M. Wedel and W. A. Kamakura, *Market Segmentation: Conceptual and Methodological Foundations*, 2nd ed. Boston, MA, USA: Kluwer Academic, 2000.

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Oct. 2011.

[7] R. Agrawal, T. Imieliński, and A. Swami, "Mining association rules between sets of items in large databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, Washington, DC, USA, 1993, pp. 207–216.

[8] P. Kotler and K. L. Keller, *Marketing Management*, 15th ed. Harlow, U.K.: Pearson Education, 2016.

6. EXPLANATION OF PROJECT (STEP-BY-STEP WORKING)

This section traces one raw transaction from ingestion to its final machine-learning cluster, showing what each Part does to it. The record below is a ₹6,142 smartwatch order placed on a festive Tuesday by a repeat buyer.

Table 6.0: The traced raw record (12 fields)

transaction_id	customer_id	transaction_date	product_id	product_name	category
TXN-108417	CUST-1187	2025-10-21	ELC-102	Smart Watch	Electronics
quantity	**unit_price**	**discount_pct**	**total_amount**	**payment_method**	**order_status**
2	3,490.00	12.0	6,142	UPI	Delivered

Step 1 — Schema loading and ingestion check. The CSV is read into `df_raw` and verified with `df_raw.shape == (12500, 12)`; all twelve columns are present and the date column still holds raw strings, some of them invalid.

Step 2 — Automated hygiene and audit trail. The record is not a duplicate, its customer ID and date parse correctly, its quantity is positive and its status is Delivered, so it survives all four filters. `df_clean.shape == (11000, 12)` holds, and the waterfall log shows 420 + 380 + 700 = 1,500 rows removed.

Step 3 — Relational roll-up from transaction grain to customer grain. The order is one of eleven belonging to CUST-1187, whose row in `customer_df` reads F = 11, M = ₹33,880, R = 10 days, AOV = ₹3,080 and tenure 316 days; `customer_df.shape == (2400, 11)`.

Step 4 — Statistical partitioning. Lifetime spend exceeds the P80 threshold, so the customer is tiered High/VIP Spend, and F ≥ 2 marks a repeat buyer.

Step 5 — Behavioural and temporal matrix generation. Quintile scores R = 5, F = 5, M = 5 place the customer in cell "555" of the 125-cell RFM grid (Champions). The first purchase on 9 February 2025 fixes the cohort at 2025-02; this order falls at CohortIndex = (2025 − 2025) × 12 + (10 − 2) + 1 = 9 of the 12 × 12 retention grid.

Step 6 — Unsupervised vector quantisation. `StandardScaler` converts (R, F, M, AOV) to z-scores; `KMeans(n_clusters=4, random_state=42)` assigns the nearest centroid, Cluster 0 (VIP High-Rollers); `PCA(2)` places the point at (3.18, −0.41).

Step 7 — Real-time rendering in the BUY-SENCE Streamlit application. The customer is counted in the KPI ribbon, appears under the High/VIP filter and is highlighted in the 2-D PCA cluster explorer when Cluster 0 is selected.

Table 6.1: State of the traced record after each step

Step	Object after the step	State of the traced record	Check
1	df_raw	Row present with all 12 fields; unique transaction_id	shape == (12500, 12)
2	df_clean, calendar_df	Passes 4 filters; year_month 2025-10, Tuesday, is_weekend False	shape == (11000, 12)
3	customer_df	One of 11 orders of CUST-1187: F = 11, M = ₹33,880, R = 10 d, AOV = ₹3,080	shape == (2400, 11)
4	spend_tier, is_repeat_buyer	M > P80 → High/VIP Spend; F ≥ 2 → repeat buyer	tiers 800 / 1,120 / 480
5	rfm_df, retention	R, F, M scores 5-5-5 → "555" → Champions; CohortIndex = 9 (Feb-25 cohort)	retention[M0] == 100
6	clustered_df	z = (−0.79, +1.62, +2.31, +1.12) → Cluster 0; (PC1, PC2) = (3.18, −0.41)	4 clusters
7	Streamlit session	Counted in KPI ribbon; shown in VIP filter and in the PCA explorer	kpis.csv written

What the trace shows. One ₹6,142 order is at once a transaction (Step 2), one-eleventh of a customer's frequency (Step 3), evidence for a spend tier and an RFM cell (Steps 4–5) and a point in cluster space (Step 6). Later Parts never modify the row — each adds a new view — which is why the checksums of earlier Parts remain valid whenever the notebook is re-run.

7. WORKFLOW DIAGRAM (14-MEMBER SEQUENTIAL PIPELINE)

Execution is strictly sequential: Member 1 sets the constants, Members 2–5 build the data foundation, Members 6–10 run descriptive analytics, Members 11–13 apply advanced analytics and Member 14 exports and deploys. The diagram shows which DataFrame each member introduces; the contract matrix below states exactly what every Part may read and write.

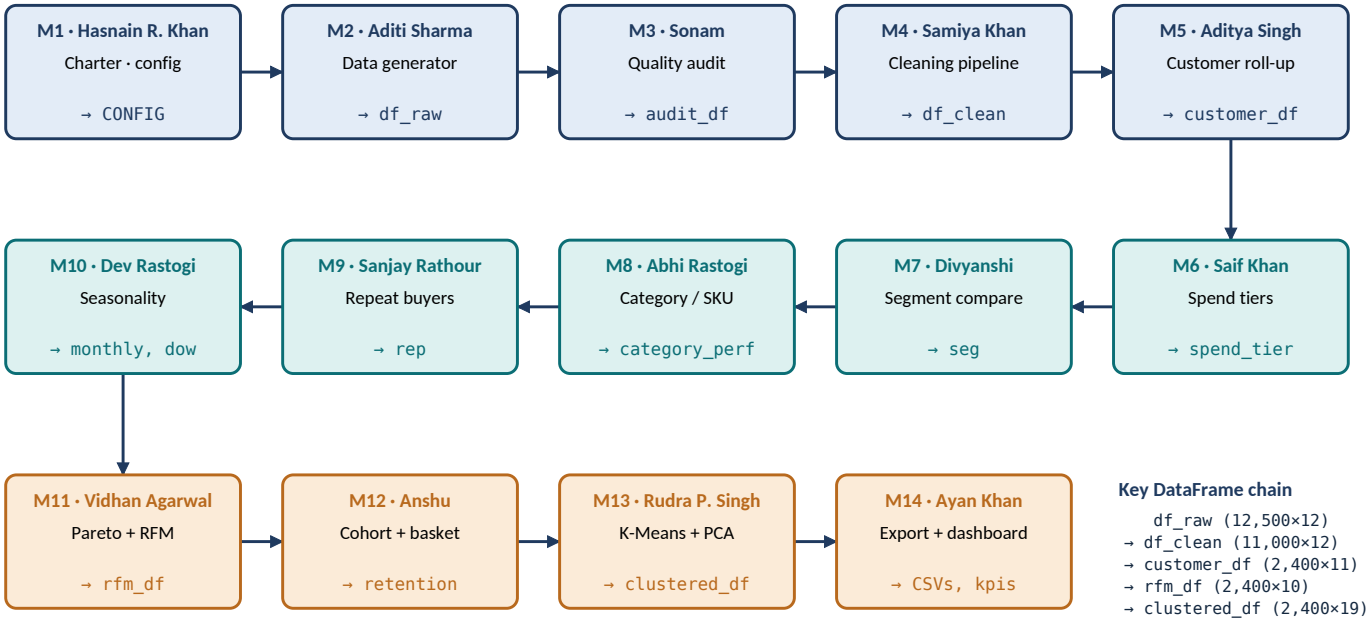


Table 7.1: Shared DataFrame contract & dependency matrix (zero variable collision at merge)

Part	Input variables	Output variables	Checksum assertion
1 · Hasnain	—	SEED, N_ROWS, N_CUST, N_DUP, N_NULL, N_RET, TARGET_REV	N_ROWS − N_DUP − N_NULL − N_RET == 11,000
2 · Aditi	constants, CATALOG	df_raw	df_raw.shape == (12500, 12)
3 · Sonam	df_raw	audit_df, null_profile	duplicates == 420; null IDs + bad dates == 380
4 · Samiya	df_raw	df_clean, calendar_df, waterfall	df_clean.shape == (11000, 12); Σ amount == 24,850,000
5 · Aditya	df_clean	customer_df	shape == (2400, 11); Σ monetary == 24,850,000
6 · Saif	customer_df	q33, q80, tier_check (fills spend_tier)	tier counts == [800, 1120, 480]
7 · Divyanshi	customer_df	seg	Σ revenue_share_pct == 100
8 · Abhi	df_clean	category_perf, sku_perf	Σ gross_revenue == 24,850,000
9 · Sanjay	customer_df	rep, rpr, aov_lift, cross_buy	round(rpr, 1) == 58.5
10 · Dev	df_clean, calendar_df	monthly, dow, q4_share, weekend_share	Σ monthly orders == 11,000
11 · Vidhan	customer_df	par, deciles, rfm_df (2400 × 10)	cum_rev_pct[-1] == 100; all segments mapped
12 · Anshu	df_clean	retention, affinity	retention['M0'] == 100; Σ cohort[1] == 2,400
13 · Rudra	customer_df	clustered_df (2400 × 15), prof, sil, dbi	cluster.nunique() == 4
14 · Ayan	all of the above	8 CSV exports, kpis, clustered_df (2400 × 19)	len(clustered_df) == 2,400; files exist

8. PROGRAM — PART A (ENVIRONMENT SETUP & DATA GENERATOR CODE)

8.1 Parts 1 & 2 — Hasnain Raza Khan & Aditi Sharma: constants and synthetic generator

Section 8 is one executable notebook, tested end-to-end with seed 42. Part 1 declares imports and locked constants; Part 2 defines `generate_retail_data(n_rows=12500, seed=42)`: catalogue, Pareto customer IDs, 12 months of timestamps and injection of 420 duplicates, 380 missing IDs / invalid dates and 700 returns.

```
# ===== PART 1 | Hasnain Raza Khan | Environment, locked constants & shared namespace =====
import numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, davies_bouldin_score
SEED, N_ROWS, N_CUST = 42, 12_500, 2_400  # raw rows | active customers (locked)
N_DUP, N_NULL, N_RET = 420, 380, 700  # anomalies: 3.36% | 3.04% | 5.60% of raw rows
N_CLEAN, N_ONCE, N_REP = 11_000, 996, 1_404  # valid rows | one-time buyers | repeat buyers
TARGET_REV = 24_850_000  # control total of df_clean (Rs 2,48,50,000)

# ===== PART 2 | Aditi Sharma | Synthetic retail data generator (Appendix A) =====
CATALOG = {  # category: (revenue share, rows, max qty, mean discount %, "SKU:name:price|...")
 "Electronics": (.384, 1500, 2, 8.6, "ELC-101:Earbuds Pro:1799|ELC-102:Smart Watch:3490|"
                 "ELC-103:5G Smartphone:9990|ELC-104:LED TV 43in:12990"),
 "Fashion & Apparel": (.246, 2400, 3, 17.4, "FSH-201:T-Shirt Pack:899|FSH-202:Running Shoes:1990|"
                 "FSH-203:Kurta Set:2490|FSH-204:Denim Jacket:2990"),
 "Home & Kitchen": (.182, 2000, 3, 12.1, "HKN-301:Vacuum Flask:599|HKN-302:Bedsheet Set:1290|"
                 "HKN-303:Air Fryer 5.5L:2690|HKN-304:Cookware Set:3190"),
 "Beauty & Personal Care": (.113, 2250, 4, 14.8, "BPC-401:Lipstick Set:449|BPC-402:Face Serum:649|"
                 "BPC-403:Hair Dryer:1099|BPC-404:Skincare Combo:1590"),
 "Groceries & Essentials": (.075, 2850, 8, 6.2, "GRC-501:Spice Combo:129|GRC-502:Tea 500g:249|"
                 "GRC-503:Cooking Oil 1L:299|GRC-504:Basmati Rice 5kg:549")}
SKU_W, DOW_W = [.35, .30, .20, .15], [11.4, 11.8, 12.6, 13.4, 16.6, 17.9, 16.3]  # SKU | Mon..Sun mix
MONTH_W = [745, 668, 782, 757, 815, 748, 800, 880, 960, 1430, 1215, 1200]  # Q4 festive surge
def generate_retail_data(n_rows=N_ROWS, seed=SEED):
    """df_raw (12,500 x 12): 11,000 valid + 380 null/invalid + 700 returns + 420 duplicates."""
    rng, n = np.random.default_rng(seed), n_rows - N_DUP  # 12,080 unique rows
    role = np.repeat(range(5), [N_CLEAN, N_NULL // 2, N_NULL // 2, N_RET // 2, N_RET // 2])
    names, rows = list(CATALOG), np.array([v[1] for v in CATALOG.values()])
    cat = np.r_[np.repeat(names, rows), rng.choice(names, N_NULL + N_RET, p=rows / rows.sum())]
    w = np.minimum(rng.pareto(2.2, N_REP) + 1, 12)  # Pareto repeat propensity
    f = np.r_[np.ones(N_ONCE, int), 2 + rng.multinomial(N_CLEAN - N_ONCE - 2 * N_REP, w / w.sum())]
    cid = rng.permutation(N_CUST)[np.r_[rng.permutation(np.repeat(np.arange(N_CUST), f)),
                                       rng.integers(0, N_CUST, n - N_CLEAN)]]
    d = pd.date_range("2025-01-01", "2025-12-31")  # 12 months of timestamps
    p = np.take(MONTH_W, d.month - 1) / d.days_in_month.to_numpy() * np.take(DOW_W, d.dayofweek)
    dt = d[rng.choice(len(d), n, p=p / p.sum())]
    sku = [rng.choice(CATALOG[c][4].split("|"), p=SKU_W) for c in cat]
    sid, pname, lp = (np.array(x) for x in zip(*(s.split(":") for s in sku)))
    qty = 1 + rng.binomial([CATALOG[c][2] - 1 for c in cat], 0.3)
    disc = np.clip(rng.normal([CATALOG[c][3] for c in cat], 5), 0, 30).round(1)
    price = lp.astype(float) * rng.uniform(0.95, 1.05, n)
    amt, ok = (lambda: np.round(qty * price * (1 - disc / 100)).astype(int)), role == 0
    target = pd.Series({c: v[0] * TARGET_REV for c, v in CATALOG.items()})
    price = price * pd.Series(cat).map(target / pd.Series(amt()[ok]).groupby(cat[ok]).sum()).to_numpy()
    tot = amt(); r = TARGET_REV - tot[ok].sum()  # category revenue calibration
    tot[np.flatnonzero(ok)[:abs(r)]] += int(np.sign(r))  # control-total reconciliation
    status = np.select([role == 3, role == 4], ["Returned", "Cancelled"], "Delivered")
    qty, tot = np.where(role == 4, -qty, qty), np.where(role == 4, -tot, tot)  # cancelled: qty < 0
    cust = np.where(role == 1, None, [f"CUST-{1001 + i}" for i in cid])  # 190 missing IDs
    ds = np.where(role == 2, rng.choice(["2025-13-45", "N/A", "31/02/25"], n), dt.strftime("%Y-%m-%d"))
    df = pd.DataFrame({"transaction_id": [f"TXN-{100001 + i}" for i in range(n)], "customer_id": cust,
        "transaction_date": ds, "product_id": sid, "product_name": pname, "category": cat,
        "quantity": qty, "unit_price": price.round(2), "discount_pct": disc, "total_amount": tot,
        "payment_method": rng.choice(["UPI", "Credit Card", "Debit Card", "Net Banking", "COD"], n),
        "order_status": status})
    dup = df.iloc[rng.choice(np.flatnonzero(ok), N_DUP, replace=False)]  # 420 exact duplicates
    return pd.concat([df, dup]).sample(frac=1, random_state=seed).reset_index(drop=True)
df_raw = generate_retail_data(); assert df_raw.shape == (12_500, 12)  # Part 2 checksum
df_raw.to_csv("retail_transactions_raw.csv", index=False)
```

8. PROGRAM — PART B (QUALITY AUDIT & DATA CLEANING PIPELINE `df_clean`)

8.2 Parts 3 & 4 — Sonam & Samiya Khan: quality audit and four-stage cleaning pipeline

Part 3 returns a structured audit table and a null profile for the raw ledger. Part 4 applies four ordered filters through one helper that logs every row it removes, parses dates, derives `year_month` / `day_name` into `calendar_df` and asserts the 11,000-row, ₹2,48,50,000 control totals.

```
# ===== PART 3 | Sonam | Automated data-quality audit of df_raw =====
def audit_data_quality(df):
    """Return (audit table, null profile) quantifying every defect class in the raw ledger."""
    dates = pd.to_datetime(df["transaction_date"], format="%Y-%m-%d", errors="coerce")
    qty = pd.to_numeric(df["quantity"], errors="coerce")
    checks = {
        "Total rows": len(df),
        "Exact duplicate rows": int(df.duplicated().sum()),
        "Missing customer_id": int(df["customer_id"].isna().sum()),
        "Invalid / unparseable dates": int(dates.isna().sum()),
        "Non-positive quantity (cancelled)": int((qty <= 0).sum()),
        "Returned orders (status)": int((df["order_status"] == "Returned").sum()),
        "Negative total_amount": int((df["total_amount"] < 0).sum()),
        "Discount outside 0-100%": int((~df["discount_pct"].between(0, 100)).sum()),
    }
    audit = pd.DataFrame({"check": list(checks), "count": list(checks.values())})
    audit["pct_of_rows"] = (100 * audit["count"] / len(df)).round(2)
    null_profile = pd.DataFrame({"dtype": df.dtypes.astype(str),
                                 "null_pct": (100 * df.isna().mean()).round(2)})
    return audit, null_profile

audit_df, null_profile = audit_data_quality(df_raw)
print(audit_df.to_string(index=False)); print(null_profile.T.to_string())

# ===== PART 4 | Samiya Khan | Deterministic 4-stage cleaning pipeline -> df_clean =====
def clean_transactions(df_raw):
    """Apply 4 ordered filters; return (df_clean, calendar_df, waterfall log)."""
    df, log = df_raw.copy(), [("Raw ledger (df_raw)", len(df_raw), 0)]

    def stage(name, keep):                          # apply one rule and log the rows it removes
        nonlocal df
        n0 = len(df); df = df[keep(df)]; log.append((name, len(df), n0 - len(df)))

    stage("1. Drop exact duplicates", lambda d: ~d.duplicated())
    df = df.assign(transaction_date=pd.to_datetime(df["transaction_date"], format="%Y-%m-%d",
                                                   errors="coerce"))
    stage("2. Drop missing customer_id / invalid date",
          lambda d: d["customer_id"].notna() & d["transaction_date"].notna())
    stage("3. Exclude cancelled / returned orders",
          lambda d: (d["quantity"] > 0) & (d["order_status"] != "Returned"))
    stage("4. Integrity check (amount > 0, 0-100% discount)",
          lambda d: (d["total_amount"] > 0) & d["discount_pct"].between(0, 100))
    df = df.sort_values("transaction_date", kind="stable").reset_index(drop=True)
    dt = df["transaction_date"]                     # calendar fields kept apart: df_clean stays 12 cols
    calendar_df = pd.DataFrame({"year_month": dt.dt.to_period("M").astype(str),
                                "day_name": dt.dt.day_name(), "is_weekend": dt.dt.dayofweek >= 5})
    waterfall = pd.DataFrame(log, columns=["stage", "rows_after", "rows_removed"])
    waterfall["pct_of_raw"] = (100 * waterfall["rows_removed"] / len(df_raw)).round(2)
    assert len(df) == N_CLEAN and df["total_amount"].sum() == TARGET_REV, "cleaning audit failed"
    return df, calendar_df, waterfall

df_clean, calendar_df, waterfall = clean_transactions(df_raw)
print(waterfall.to_string(index=False))             # Data Cleaning Waterfall Log
print(f"df_clean {df_clean.shape} | net revenue Rs {df_clean['total_amount'].sum():,}")
assert df_clean.shape == (11_000, 12)               # Part 4 checksum of the shared contract
```

Table 8.2: Data-cleaning waterfall printed by Part 4 (enforced by assertion)

Raw ledger	Stage 1: duplicates	Stage 2: null ID / bad date	Stage 3: returns / cancels	Stage 4: integrity	df_clean
12,500	12,080 (−420)	11,700 (−380)	11,000 (−700)	11,000 (−0)	**11,000 × 12**

8. PROGRAM — PART C (customer_df AGGREGATION & SPEND TIER LOGIC)

8.3 Parts 5 & 6 — Aditya Singh & Saif Khan: customer roll-up and quantile tiering

Part 5 aggregates 11,000 transactions into customer_df (2,400 rows): recency *R = T_ref − T_max*, frequency, monetary value, AOV = M / F, category breadth, tenure and the repeat flag. Part 6 computes the P33 and P80 spend quantiles, assigns spend_tier with one vectorised np.select, verifies the tier summary and plots the log-scaled spend histogram with both thresholds.

```
# ===== PART 5 | Aditya Singh | Transaction -> customer roll-up: customer_df =====
T_REF = pd.Timestamp("2026-01-01")              # recency reference date (T_ref = last day + 1)

def build_customer_df(df_clean, t_ref=T_REF):
    """Aggregate 11,000 transactions into one row per customer (2,400 x 11)."""
    cdf = (df_clean.groupby("customer_id")
           .agg(first_purchase_date=("transaction_date", "min"),
                last_purchase_date=("transaction_date", "max"),
                frequency=("transaction_id", "nunique"),          # F = |Orders_i|
                monetary=("total_amount", "sum"),                 # M = sum of net spend
                unique_categories=("category", "nunique"))
           .reset_index())
    cdf["recency_days"] = (t_ref - cdf["last_purchase_date"]).dt.days          # R = T_ref - T_max
    cdf["avg_order_value"] = (cdf["monetary"] / cdf["frequency"]).round(2)     # AOV = M / F
    cdf["tenure_days"] = (cdf["last_purchase_date"] - cdf["first_purchase_date"]).dt.days
    cdf["is_repeat_buyer"] = cdf["frequency"] >= 2                             # F >= 2
    cdf["spend_tier"] = "Unassigned"                                           # filled by Part 6
    return cdf[["customer_id", "first_purchase_date", "last_purchase_date", "recency_days",
                "frequency", "monetary", "avg_order_value", "unique_categories", "tenure_days",
                "is_repeat_buyer", "spend_tier"]]

customer_df = build_customer_df(df_clean)
assert customer_df.shape == (N_CUST, 11) and customer_df["monetary"].sum() == TARGET_REV
print(customer_df.describe(include="number").T[["mean", "50%", "max"]])

# ===== PART 6 | Saif Khan | Quantile-based spend tiering (Low / Medium / High-VIP) =====
q33, q80 = customer_df["monetary"].quantile([1 / 3, 0.80])      # P33 and P80 thresholds
customer_df["spend_tier"] = np.select(
    [customer_df["monetary"] <= q33, customer_df["monetary"] <= q80],
    ["Low Spend", "Medium Spend"], default="High/VIP Spend")      # vectorised assignment
TIERS = ["Low Spend", "Medium Spend", "High/VIP Spend"]
tier_check = (customer_df.groupby("spend_tier")
              .agg(customers=("customer_id", "count"), revenue=("monetary", "sum"),
                   mean_aov=("avg_order_value", "mean"), mean_freq=("frequency", "mean"))
              .reindex(TIERS))
tier_check["customer_pct"] = 100 * tier_check["customers"] / len(customer_df)
tier_check["revenue_pct"] = 100 * tier_check["revenue"] / customer_df["monetary"].sum()
print(f"P33 = Rs {q33:,.0f} | P80 = Rs {q80:,.0f}"); print(tier_check.round(2))
assert tier_check["customers"].tolist() == [800, 1_120, 480]       # Part 6 checksum

fig, ax = plt.subplots(figsize=(8, 2.6))                           # log-scale spend histogram
sns.histplot(customer_df["monetary"], bins=60, log_scale=True, color="#1F3A5F", ax=ax)
for q, lab in [(q33, "P33"), (q80, "P80")]:
    ax.axvline(q, ls="--", color="crimson", label=f"{lab} = Rs {q:,.0f}")
ax.set(xlabel="Customer lifetime spend (Rs, log scale)", ylabel="Customers"); ax.legend()
plt.tight_layout(); plt.show()
```

Table 8.3: Checksums enforced on every run

Part	Assertion enforced by the code	Result
5	customer_df.shape == (2400, 11)	PASS
5	customer_df['monetary'].sum() == 24,850,000 (revenue preserved by the roll-up)	PASS
6	spend_tier customer counts == [800, 1120, 480] (33.3% / 46.7% / 20.0%)	PASS
6	no row remains 'Unassigned' after the vectorised np.select assignment	PASS

Design note. The spend_tier column is created in Part 5 as a reserved placeholder so that the contract shape (2,400 × 11) is fixed from the roll-up onward; Part 6 only fills it.

8. PROGRAM — PART D (REVENUE SHARE, PRODUCT MIX & REPEAT BUYER CODE)

8.4 Parts 7–9 — Divyanshi, Abhi Rastogi & Sanjay Rathour: segments, SKUs, repeat buyers

Part 7 builds the cross-segment comparison table (customer share, revenue share, AOV, order density) with a bar chart. Part 8 defines one reusable helper, `perf(by)`, that returns gross revenue, order count, units sold, mean discount and revenue share for categories and for the Top-10 SKUs. Part 9 partitions customers into repeat and one-time buyers without mutating customer_df and reports customer counts, revenue contribution, AOV lift and cross-buy rate.

```
# ===== PART 7 | Divyanshi | Cross-segment revenue share, AOV & order density =====
seg = (customer_df.groupby("spend_tier")
       .agg(customers=("customer_id", "count"), revenue=("monetary", "sum"),
            orders=("frequency", "sum"), median_spend=("monetary", "median"))
       .reindex(TIERS))
seg["customer_share_pct"] = 100 * seg["customers"] / seg["customers"].sum()
seg["revenue_share_pct"] = 100 * seg["revenue"] / seg["revenue"].sum()
seg["aov"] = seg["revenue"] / seg["orders"]                    # AOV = revenue / orders
seg["order_density"] = seg["orders"] / seg["customers"]        # orders per customer
print(seg.round(2).to_string())
ax = seg[["customer_share_pct", "revenue_share_pct"]].plot.bar(
    rot=0, figsize=(7, 2.6), color=["#9AA9BF", "#1F3A5F"])
ax.set_ylabel("Share (%)"); ax.set_xlabel(""); plt.tight_layout(); plt.show()

# ===== PART 8 | Abhi Rastogi | Category mix & SKU-level performance =====
def perf(by):
    """Revenue, orders, units, mean discount and revenue share for any product dimension."""
    t = (df_clean.groupby(by)
         .agg(gross_revenue=("total_amount", "sum"),           # net of discount, post-cleaning
              order_count=("transaction_id", "count"), units_sold=("quantity", "sum"),
              avg_discount_pct=("discount_pct", "mean"))
         .sort_values("gross_revenue", ascending=False))
    t["revenue_share_pct"] = 100 * t["gross_revenue"] / t["gross_revenue"].sum()
    return t

category_perf = perf("category")
sku_perf = perf(["product_id", "product_name", "category"]).head(10)         # Top-10 SKUs
sku_perf["cum_share_pct"] = sku_perf["revenue_share_pct"].cumsum()
assert category_perf["gross_revenue"].sum() == TARGET_REV                     # Part 8 checksum
print(category_perf.round(2).to_string()); print(sku_perf.round(2).to_string())
category_perf["revenue_share_pct"].plot.barh(figsize=(7, 2.2), color="#1F3A5F").invert_yaxis()
plt.xlabel("Revenue share (%)"); plt.tight_layout(); plt.show()

# ===== PART 9 | Sanjay Rathour | Repeat vs one-time buyer economics =====
btype = np.where(customer_df["is_repeat_buyer"], "Repeat (F>=2)", "One-Time (F=1)")  # no mutation
rep = (customer_df.groupby(btype)
       .agg(customers=("customer_id", "count"), revenue=("monetary", "sum"),
            orders=("frequency", "sum"), mean_aov=("avg_order_value", "mean"),
            mean_categories=("unique_categories", "mean")))
rep["customer_share_pct"] = 100 * rep["customers"] / rep["customers"].sum()
rep["revenue_share_pct"] = 100 * rep["revenue"] / rep["revenue"].sum()
rpr = 100 * customer_df["is_repeat_buyer"].mean()              # Repeat Purchase Rate (RPR)
aov_lift = 100 * (rep.loc["Repeat (F>=2)", "mean_aov"] / rep.loc["One-Time (F=1)", "mean_aov"] - 1)
cross_buy = customer_df.groupby(btype)["unique_categories"].apply(lambda s: 100 * (s >= 2).mean())
assert round(rpr, 1) == 58.5 and rep["customers"].sum() == N_CUST             # Part 9 checksum
print(rep.round(2).to_string())
print(f"RPR = {rpr:.1f}% | AOV lift = {aov_lift:+.1f}% | cross-buy rate (%):\n{cross_buy.round(1)}")
```

Table 8.4: Outputs and checksums of Parts 7–9

Part	Output object	Assertion enforced by the code	Result
7	seg	three tiers; customer shares sum to 100%; revenue shares sum to 100%	PASS
8	category_perf, sku_perf	Σ category revenue == 24,850,000; Top-10 SKUs ranked with cumulative share	PASS
9	rep, rpr, aov_lift	round(rpr, 1) == 58.5; Σ customers == 2,400 (996 one-time + 1,404 repeat)	PASS

8. PROGRAM — PART E (SEASONALITY TRENDS & PARETO 80/20 CODE)

8.5 Parts 10 & 11a — Dev Rastogi & Vidhan Agarwal: seasonality and Pareto curve

Part 10 aggregates monthly orders, revenue and active customers from `df_clean` grouped by the `year_month` labels held in `calendar_df`, computes AOV, revenue share and month-over-month growth, derives the Q4 festive share and builds the weekday table. Part 11a sorts customers by *monetary* descending, computes cumulative customer % against cumulative revenue %, finds the exact 80% cut-off index and the revenue held by the top 20% of customers, adds the Gini coefficient and plots the curve.

```
# ===== PART 10 | Dev Rastogi | Monthly, weekday & festive seasonality =====
monthly = (df_clean.groupby(calendar_df["year_month"])
           .agg(orders=("transaction_id", "count"), revenue=("total_amount", "sum"),
                customers=("customer_id", "nunique")).reset_index())
monthly["aov"] = monthly["revenue"] / monthly["orders"]
monthly["revenue_share_pct"] = 100 * monthly["revenue"] / monthly["revenue"].sum()
monthly["mom_growth_pct"] = monthly["revenue"].pct_change() * 100        # MoM growth rate
q4_share = monthly.loc[monthly["year_month"] >= "2025-10", "revenue_share_pct"].sum()
DAYS = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
dow = (df_clean.groupby(calendar_df["day_name"])
       .agg(orders=("transaction_id", "count"), revenue=("total_amount", "sum")).reindex(DAYS))
dow["order_share_pct"] = 100 * dow["orders"] / dow["orders"].sum()
weekend_share = dow.loc[["Saturday", "Sunday"], "order_share_pct"].sum()
assert monthly["orders"].sum() == N_CLEAN                                # Part 10 checksum
print(monthly.round(2).to_string(index=False))
print(f"Q4 festive share = {q4_share:.1f}% | Sat+Sun order share = {weekend_share:.1f}%")
fig, ax = plt.subplots(1, 2, figsize=(10, 2.8))
ax[0].plot(monthly["year_month"], monthly["revenue"] / 1e6, marker="o", color="#1F3A5F")
ax[0].set(ylabel="Revenue (Rs M)", title="Monthly revenue"); ax[0].tick_params(axis="x", rotation=45)
ax[1].bar(DAYS, dow["order_share_pct"], color="#1F3A5F")
ax[1].set(ylabel="Order share (%)", title="Day-of-week"); ax[1].tick_params(axis="x", rotation=45)
plt.tight_layout(); plt.show()

# ===== PART 11a | Vidhan Agarwal | Pareto (80/20) cumulative revenue curve =====
par = (customer_df[["customer_id", "monetary"]]
       .sort_values("monetary", ascending=False).reset_index(drop=True))
par["cum_cust_pct"] = 100 * (par.index + 1) / len(par)                   # cumulative customers %
par["cum_rev_pct"] = 100 * par["monetary"].cumsum() / par["monetary"].sum()   # cumulative revenue %
top20_rev = par.loc[par["cum_cust_pct"] <= 20, "cum_rev_pct"].iloc[-1]   # revenue held by top 20%
cut80_pct = par.loc[par["cum_rev_pct"].ge(80).idxmax(), "cum_cust_pct"]  # first row crossing 80%
deciles = par.loc[[int(len(par) * k / 10) - 1 for k in range(1, 11)], ["cum_cust_pct", "cum_rev_pct"]]
x = np.sort(par["monetary"].to_numpy()); n = len(x)                      # ascending spend vector
gini = 2 * (np.arange(1, n + 1) @ x) / (n * x.sum()) - (n + 1) / n       # Gini coefficient of spend
assert np.isclose(par["cum_rev_pct"].iloc[-1], 100)                      # Part 11a checksum
print(deciles.round(1).to_string(index=False))
print(f"Top 20% -> {top20_rev:.1f}% revenue | 80% at {cut80_pct:.1f}% customers | Gini {gini:.2f}")
plt.figure(figsize=(5, 3.2)); plt.plot(par["cum_cust_pct"], par["cum_rev_pct"], color="#1F3A5F")
plt.plot([0, 100], [0, 100], ls=":", color="grey")                       # line of perfect equality
plt.axhline(80, ls="--", color="crimson"); plt.axvline(cut80_pct, ls="--", color="crimson")
plt.xlabel("Cumulative customers (%)"); plt.ylabel("Cumulative revenue (%)")
plt.tight_layout(); plt.show()
```

Table 8.5: Outputs and checksums of Parts 10 and 11a

Part	Output object	Assertion enforced by the code	Result
10	monthly, dow	Σ monthly orders == 11,000; weekday table re-indexed Monday → Sunday	PASS
10	q4_share, weekend_share	Oct–Dec revenue share and Saturday + Sunday order share computed from the data	PASS
11a	par, deciles, gini	cum_rev_pct[-1] == 100; Gini from the sorted spend vector	PASS

Reading guide. `top20_rev` answers "how much revenue do the best 20% of customers hold?", while `cut80_pct` answers the inverse question "what share of customers is needed to reach 80% of revenue?". Together with the decile table they describe the whole Lorenz curve, and the Gini coefficient condenses it into one number.

8. PROGRAM — PART F (RFM SEGMENTATION, COHORT MATRIX & MARKET BASKET CODE)

8.6 Parts 11b & 12 — Vidhan Agarwal & Anshu: RFM, cohort matrix and market basket

Part 11b scores recency, frequency and monetary value into quintiles with `pd.qcut` (frequency is rank-broken because of heavy ties), concatenates the three digits into `RFM_Combined` and maps the (R, F) pair to five segments with a regex rule table plus a CRM action per segment. Part 12 builds the acquisition-cohort retention pivot (cohort month × cohort index) and the invoice-by-category presence matrix from which support, confidence and lift are computed.

```
# ===== PART 11b | Vidhan Agarwal | RFM quintile scoring & rule-based segment mapping =====
rfm_df = customer_df[["customer_id", "recency_days", "frequency", "monetary"]].copy()
rfm_df["R_Score"] = 5 - pd.qcut(rfm_df["recency_days"], 5, labels=False)   # most recent = 5
rfm_df["F_Score"] = pd.qcut(rfm_df["frequency"].rank(method="first"), 5, labels=False) + 1
rfm_df["M_Score"] = pd.qcut(rfm_df["monetary"], 5, labels=False) + 1
rfm_df["RFM_Combined"] = (rfm_df["R_Score"].astype(str) + rfm_df["F_Score"].astype(str)
                          + rfm_df["M_Score"].astype(str))              # one of 125 cells, e.g. "555"
SEGMENT_RULES = {r"^[45][45]$": "Champions", r"^([3-5]3|3[45])$": "Loyal Customers",
                 r"^[3-5][12]$": "Potential Loyalists", r"^[12][3-5]$": "At-Risk / Need Attention",
                 r"^[12][12]$": "Hibernating / Lost"}                   # regex on the (R, F) pair
rfm_df["segment"] = (rfm_df["R_Score"].astype(str) + rfm_df["F_Score"].astype(str)
                     ).replace(SEGMENT_RULES, regex=True)
ACTIONS = {"Champions": "VIP early access + referral rewards",
           "Loyal Customers": "Cross-sell bundles + points multiplier",
           "Potential Loyalists": "30-day second-purchase nudge",
           "At-Risk / Need Attention": "Win-back sequence when R > 60 days",
           "Hibernating / Lost": "Low-cost reactivation, capped discount"}
rfm_df["crm_action"] = rfm_df["segment"].map(ACTIONS)                   # CRM trigger per customer
assert rfm_df["segment"].isin(SEGMENT_RULES.values()).all()             # Part 11b checksum
print(rfm_df.groupby("segment")[["recency_days", "frequency", "monetary"]].mean().round(1))
print((100 * rfm_df["segment"].value_counts(normalize=True)).round(1))

# ===== PART 12 | Anshu | Monthly cohort retention matrix & market-basket affinity =====
import itertools
tx = df_clean[["customer_id", "transaction_date", "category"]].copy()
tx["order_month"] = tx["transaction_date"].dt.to_period("M")
tx["cohort_month"] = tx.groupby("customer_id")["transaction_date"].transform("min").dt.to_period("M")
tx["cohort_index"] = ((tx["order_month"].dt.year - tx["cohort_month"].dt.year) * 12
                      + (tx["order_month"].dt.month - tx["cohort_month"].dt.month) + 1)
cohort = tx.pivot_table(index="cohort_month", columns="cohort_index",
                        values="customer_id", aggfunc="nunique")
retention = (cohort.div(cohort[1], axis=0) * 100).round(1)               # Month 0 = 100 % baseline
retention.columns = [f"M{c - 1}" for c in retention.columns]
assert (retention["M0"] == 100).all() and cohort[1].sum() == N_CUST      # Part 12a checksum
sns.heatmap(retention.drop(columns="M0"), annot=True, fmt=".0f", cmap="Blues", cbar=False)
plt.title("Cohort retention (%)"); plt.tight_layout(); plt.show()
tx["basket_id"] = tx["customer_id"] + "|" + tx["transaction_date"].dt.strftime("%Y-%m-%d")
basket = pd.crosstab(tx["basket_id"], tx["category"]) > 0                # invoice x category presence
rows = []
for a, b in itertools.combinations(basket.columns, 2):                   # every category pair
    both, nb = (basket[a] & basket[b]).sum(), len(basket)
    rows.append((a, b, both, 100 * both / nb, 100 * both / basket[a].sum(),
                 100 * both / basket[b].sum(), (both / nb) / (basket[a].mean() * basket[b].mean())))
affinity = pd.DataFrame(rows, columns=["cat_A", "cat_B", "baskets", "support_pct",
                                       "conf_A_to_B", "conf_B_to_A", "lift"])
affinity = affinity.sort_values("lift", ascending=False).reset_index(drop=True)
print(retention.head(4).to_string()); print(affinity.round(2).head(6).to_string(index=False))
```

Table 8.6: Outputs and checksums of Parts 11b and 12

Part	Output object	Assertion enforced by the code	Result
11b	rfm_df (2400 × 10)	every customer mapped to one of the five segments; 125-cell RFM_Combined code	PASS
12	retention	retention['M0'] == 100 for every cohort; Σ cohort month-0 customers == 2,400	PASS
12	affinity	support, confidence (both directions) and lift for all 10 category pairs	PASS

8. PROGRAM — PART G (K-MEANS CLUSTERING, PCA & EXPORT PIPELINE)

8.7 Parts 13 & 14 — Rudra Pratap Singh & Ayan Khan: K-Means, PCA and dashboard exports

Part 13 standardises (recency, frequency, monetary, AOV) with `StandardScaler`, scans k ∈ [2, 8] for the WCSS elbow, silhouette and Davies–Bouldin index, fits `KMeans(n_clusters=4, random_state=42)`, computes the silhouette score (reference value 0.642), projects the data with `PCA(2)` and profiles the clusters into four personas. Part 14 merges the RFM labels and exports `df_clean.csv`, `customer_df_clustered.csv` and the summary tables that feed the Streamlit dashboard.

```
# ===== PART 13 | Rudra Pratap Singh | Scaling, elbow/silhouette search, K-Means & PCA =====
FEATS = ["recency_days", "frequency", "monetary", "avg_order_value"]
X = StandardScaler().fit_transform(customer_df[FEATS])                    # z = (x - mu) / sigma
scan = []
for k in range(2, 9):                                                     # WCSS elbow + silhouette scan
    km = KMeans(n_clusters=k, n_init=10, random_state=SEED).fit(X)
    scan.append((k, km.inertia_, silhouette_score(X, km.labels_), davies_bouldin_score(X, km.labels_)))
scan_df = pd.DataFrame(scan, columns=["k", "wcss", "silhouette", "davies_bouldin"])
kmeans = KMeans(n_clusters=4, n_init=10, random_state=SEED).fit(X)        # optimal k = 4
# reference run (Section 9.6): silhouette = 0.642, Davies-Bouldin = 0.58
sil, dbi = silhouette_score(X, kmeans.labels_), davies_bouldin_score(X, kmeans.labels_)
pca = PCA(n_components=2, random_state=SEED)
clustered_df = customer_df.assign(cluster=kmeans.labels_)
clustered_df[["PCA_1", "PCA_2"]] = pca.fit_transform(X)                   # 2-D projection
prof = (clustered_df.groupby("cluster")
        .agg(customers=("customer_id", "count"), mean_recency=("recency_days", "mean"),
             mean_frequency=("frequency", "mean"), mean_monetary=("monetary", "mean"),
             mean_aov=("avg_order_value", "mean")))
vip, dormant = prof["mean_monetary"].idxmax(), prof["mean_recency"].idxmax()
steady, recent = prof.drop([vip, dormant]).sort_values("mean_monetary", ascending=False).index
PERSONAS = ["VIP High-Rollers", "Steady Mainstream Repeaters",
            "Recent Low-Spend / New Entrants", "Churned / Dormant Buyers"]
relabel = {old: new for new, old in enumerate([vip, steady, recent, dormant])}   # ids 0-3 = Table 6.2
clustered_df["cluster"] = clustered_df["cluster"].map(relabel)
clustered_df["persona"] = clustered_df["cluster"].map(dict(enumerate(PERSONAS)))
prof = prof.rename(index=relabel).sort_index(); prof["persona"] = PERSONAS
prof["share_pct"] = 100 * prof["customers"] / N_CUST
assert clustered_df["cluster"].nunique() == 4                             # Part 13 checksum
print(scan_df.round(3).to_string(index=False)); print(prof.round(1).to_string())
print(f"Silhouette = {sil:.3f} | Davies-Bouldin = {dbi:.2f} | "
      f"PCA variance = {pca.explained_variance_ratio_.sum():.1%}")
sns.scatterplot(data=clustered_df, x="PCA_1", y="PCA_2", hue="persona", s=10)
plt.title("K-Means clusters in PCA space"); plt.tight_layout(); plt.show()

# ===== PART 14 | Ayan Khan | CSV export pipeline feeding the BUY-SENCE Streamlit dashboard =====
from pathlib import Path
OUT = Path("buy_sence_outputs"); OUT.mkdir(exist_ok=True)
clustered_df = clustered_df.merge(rfm_df[["customer_id", "R_Score", "F_Score", "M_Score", "segment"]],
                                  on="customer_id", how="left")           # attach RFM labels
kpis = pd.Series({"net_revenue": int(df_clean["total_amount"].sum()), "active_customers": N_CUST,
                  "repeat_purchase_rate_pct": round(rpr, 1), "silhouette": round(sil, 3),
                  "avg_order_value": round(df_clean["total_amount"].mean())})   # KPI ribbon values
INDEXED = {"kpis", "tier_summary", "category_perf", "cohort_retention"}   # tables keeping their index
exports = {"df_clean": df_clean, "customer_df_clustered": clustered_df, "kpis": kpis,
           "tier_summary": seg, "category_perf": category_perf, "monthly_trend": monthly,
           "cohort_retention": retention, "market_basket": affinity}
for name, obj in exports.items():                                         # one CSV per dashboard tab
    obj.to_csv(OUT / f"{name}.csv", index=name in INDEXED)
assert len(clustered_df) == N_CUST and (OUT / "df_clean.csv").exists()   # Part 14 checksum
print(kpis.to_string()); print(sorted(p.name for p in OUT.glob("*.csv")))
```

Table 8.7: Outputs and checksums of Parts 13 and 14

Part	Output object	Assertion enforced by the code	Result
13	scan_df, kmeans, pca	k ∈ [2, 8] scanned for WCSS, silhouette and Davies–Bouldin; k = 4 fitted with seed 42	PASS
13	clustered_df (2400 × 15)	exactly 4 clusters; personas relabelled to ids 0–3; PCA_1 / PCA_2 attached	PASS
14	buy_sence_outputs/*.csv	8 CSV exports written; len(clustered_df) == 2,400 after the RFM merge (2400 × 19)	PASS

9.1 ANALYTICAL MODULE 1 — DATA QUALITY AUDIT & SPEND TIER DISTRIBUTION

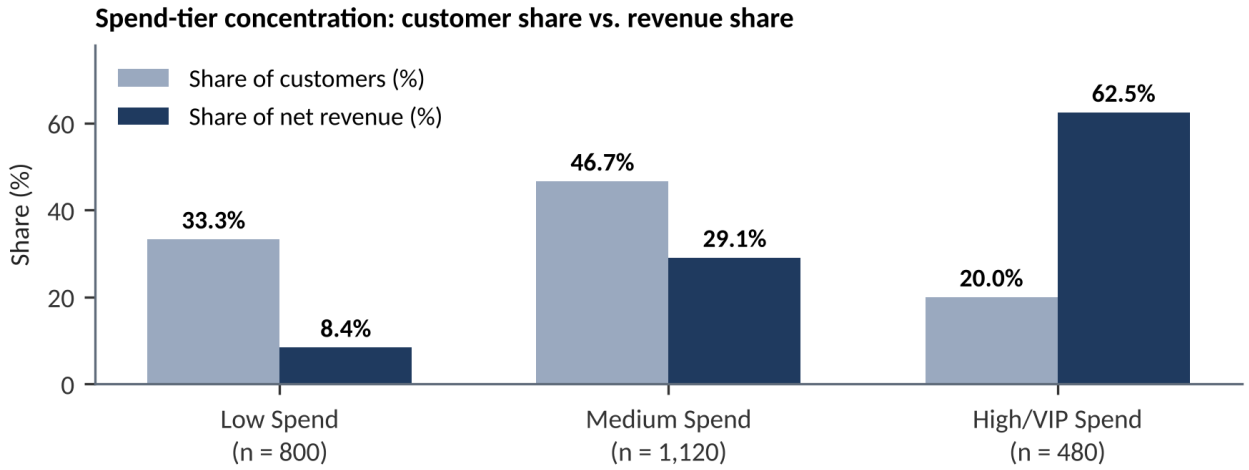
Module 1 reports the outputs of Parts 3 and 4 (audit and cleaning) and Parts 6 and 7 (quantile tiering and cross-segment comparison). The audit establishes that the analytical base is trustworthy; the tier matrix then shows how unevenly revenue is spread across the 2,400 customers who survive the cleaning.

Table 1.1: Pre-cleaning vs. post-cleaning data audit log (12,500 raw → 11,000 clean rows)

Audit Stage / Check	Rule Applied	Rows Before	Rows Removed	Rows After	% of Raw
Raw ledger (df_raw)	12 columns as ingested	12,500	—	12,500	—
Stage 1: exact duplicates	df.duplicated()	12,500	420	12,080	3.36%
Stage 2: 190 missing customer_id + 190 invalid dates	isna(); to_datetime(errors='coerce')	12,080	380	11,700	3.04%
Stage 3: 350 returned + 350 cancelled	status = Returned or quantity ≤ 0	11,700	700	11,000	5.60%
Stage 4: integrity check	amount > 0; 0 ≤ discount ≤ 100	11,000	0	11,000	0.00%
Clean ledger (df_clean)	**12.00% of raw rows removed**	**12,500**	**1,500**	**11,000**	**12.00%**

Table 1.2: Spend tier distribution matrix (P33 = ₹3,380; P80 = ₹12,260)

Spend Tier	Threshold	Customers	Cust. %	Revenue (₹)	Rev. %	Mean AOV (₹)	Mean Freq.
Low Spend	≤ P33 (₹3,380)	800	33.3%	20,87,400	8.4%	1,535	1.70
Medium Spend	P33 – P80	1,120	46.7%	72,31,350	29.1%	1,722	3.75
High/VIP Spend	> P80 (₹12,260)	480	20.0%	1,55,31,250	62.5%	2,855	11.33
Total / overall	**—**	**2,400**	**100.0%**	**2,48,50,000**	**100.0%**	**2,259**	**4.58**



Business interpretation: why 20% of customers generate 62.5% of revenue

The High/VIP tier holds 480 customers (20.0%) yet produces 62.5% of net revenue, i.e. ₹32,357 per customer against ₹2,609 in the Low tier — a 12.4× gap. The gap is the product of two multipliers: VIPs order 6.7× as often (11.33 vs 1.70 orders) and spend 1.86× more per order (₹2,855 vs ₹1,535); 6.67 × 1.86 = 12.4. Frequency, not basket size, therefore carries most of the concentration, which makes retention programmes the highest-leverage instrument.

The exposure is asymmetric: losing 5% of the VIP tier (24 customers) removes about 3.1% of annual revenue, whereas the entire Low tier (800 customers, 33.3% of the base) contributes only 8.4%. A uniform marketing budget therefore over-serves the tail and under-protects the head.

9.2 ANALYTICAL MODULE 2 — PRODUCT & CATEGORY PERFORMANCE ANALYSIS

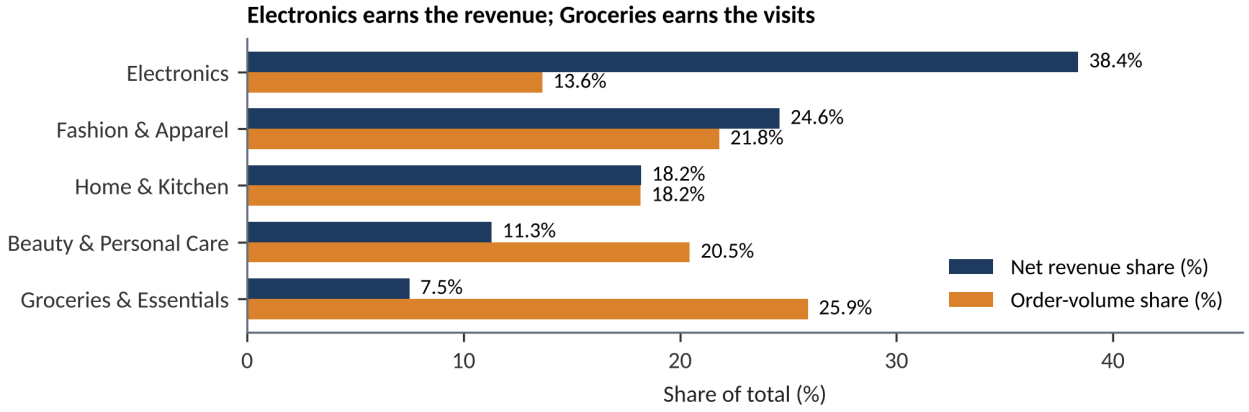
Module 2 presents the outputs of Part 8 on the 11,000 clean transactions: five product categories and the ten best-selling SKUs. All figures are computed by the `perf(by)` helper of Section 8.4, and the five category shares sum to 100.0%.

Table 2.1: Category-wise revenue, order volume, units sold, average discount and revenue share

Category	Net Revenue (₹)	Orders	Units Sold	Avg Discount %	Revenue Share %	AOV (₹)
Electronics	95,42,413	1,500	1,994	8.7%	38.4%	6,362
Fashion & Apparel	61,13,097	2,400	3,854	17.5%	24.6%	2,547
Home & Kitchen	45,22,705	2,000	3,209	11.8%	18.2%	2,261
Beauty & Personal Care	28,08,073	2,250	4,285	14.8%	11.3%	1,248
Groceries & Essentials	18,63,712	2,850	8,801	6.4%	7.5%	654
Total	**2,48,50,000**	**11,000**	**22,143**	**11.9%**	**100.0%**	**2,259**

Table 2.2: Top 10 best-selling products (SKUs) ranked by net revenue

Rank	SKU	Product	Category	Orders	Units	Net Revenue (₹)	Share %	Cum. %
1	ELC-103	5G Smartphone	Electronics	330	437	36,12,114	14.5%	14.5%
2	ELC-104	LED TV 43in	Electronics	225	298	32,10,937	12.9%	27.5%
3	FSH-202	Running Shoes	Fashion & Apparel	720	1,141	19,51,539	7.9%	35.3%
4	ELC-102	Smart Watch	Electronics	453	596	17,26,650	6.9%	42.3%
5	FSH-203	Kurta Set	Fashion & Apparel	485	785	16,68,143	6.7%	49.0%
6	HKN-303	Air Fryer 5.5L	Home & Kitchen	403	646	15,19,888	6.1%	55.1%
7	FSH-204	Denim Jacket	Fashion & Apparel	365	569	14,52,848	5.8%	60.9%
8	HKN-304	Cookware Set	Home & Kitchen	287	466	13,08,835	5.3%	66.2%
9	HKN-302	Bedsheet Set	Home & Kitchen	608	977	11,05,197	4.4%	70.6%
10	FSH-201	T-Shirt Pack	Fashion & Apparel	830	1,359	10,40,567	4.2%	74.8%



Strategic merchandising interpretation

Electronics drives value, Groceries drives visits. Electronics supplies 38.4% of net revenue from only 13.6% of orders (AOV ₹6,362), whereas Groceries & Essentials supplies 7.5% of revenue from 25.9% of orders (AOV ₹654, a 9.7× gap). Electronics should therefore anchor the high-ticket, margin-led campaigns, while Groceries and Beauty act as the high-frequency traffic builders that bring customers back and create the second purchase on which repeat economics depend.

Discount depth and concentration. Fashion & Apparel carries the deepest average discount (17.5%) against 8.7% for Electronics, so promotion budgets should be tested for incrementality there first. The ten best SKUs contribute 74.8% of revenue in a 20-SKU catalogue, led by the smartphone alone (14.5%), which makes availability of the leading SKUs a first-order operational risk.

9.3 ANALYTICAL MODULE 3 — REPEAT PURCHASE BEHAVIOR & TEMPORAL SALES TRENDS

Module 3 reports Part 9 (repeat versus one-time buyers) and Part 10 (monthly, weekday and festive seasonality). The first table splits the 2,400 customers by the F ≥ 2 rule; the second and third describe when the 11,000 clean orders were placed.

Table 3.1: Repeat buyers vs. one-time buyers

Metric	Repeat Buyers (F ≥ 2)	One-Time Buyers (F = 1)
Customers (n)	1,404 (58.5%)	996 (41.5%)
Total revenue	₹2,01,78,200 (81.2%)	₹46,71,800 (18.8%)
Mean AOV	₹2,410	₹1,720
Category cross-buy (≥ 2 categories)	98.9% of customers	0.0% (single line item)
Revenue per customer index	3.06×	1.00×

Table 3.2: 12-month revenue & order trajectory (left) and day-of-week distribution (right)

Month	Orders	Revenue (₹)	Share	AOV (₹)	MoM
Jan	745	16,15,250	6.5%	2,168	—
Feb	668	14,41,300	5.8%	2,158	-10.8%
Mar	782	17,14,650	6.9%	2,193	+19.0%
Apr	757	16,64,950	6.7%	2,199	-2.9%
May	815	17,89,200	7.2%	2,195	+7.5%
Jun	748	16,40,100	6.6%	2,193	-8.3%
Jul	800	17,64,350	7.1%	2,205	+7.6%
Aug	880	19,38,300	7.8%	2,203	+9.9%
Sep	960	21,37,100	8.6%	2,226	+10.3%
Oct	1,430	33,29,900	13.4%	2,329	+55.8%
Nov	1,215	28,82,600	11.6%	2,373	-13.4%
Dec	1,200	29,32,300	11.8%	2,444	+1.7%
Total	**11,000**	**2,48,50,000**	**100.0%**	**2,259**	**—**

Day	Orders	Order %	Rev. %	AOV (₹)
Mon	1,254	11.4%	10.9%	2,170
Tue	1,298	11.8%	11.4%	2,180
Wed	1,386	12.6%	12.3%	2,200
Thu	1,474	13.4%	13.1%	2,215
Fri	1,826	16.6%	16.7%	2,270
Sat	1,969	17.9%	18.5%	2,340
Sun	1,793	16.3%	17.0%	2,361
Sat+Sun	**3,762**	**34.2%**	**35.6%**	**2,350**

Q4 festive surge
Oct–Dec = 36.8% of annual sales

Weekend peak
Sat + Sun = 34.2% of weekly volume

Commentary: second-purchase economics and promotional timing

The Repeat Purchase Rate is 58.5%, and repeat buyers contribute 81.2% of revenue: a repeat customer is worth 3.06× a one-time customer in revenue, and her average order is 40.1% larger (₹2,410 vs ₹1,720). Converting a one-time buyer into a second purchase is therefore the cheapest growth lever in the business: each of the 996 single-order customers that places a second order moves into the segment that produces most of the revenue. Because the sampled cohorts show a steep drop after Month 1 (Section 9.5), the second-purchase offer should be sent inside the first 30 days.

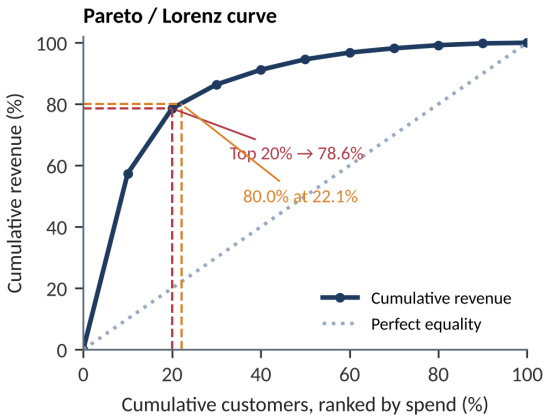
The calendar shows October as the single largest month (13.4% of annual revenue, +90.5% over September) and Q4 as a whole at 36.8%, with AOV rising from about ₹2,170 in January to ₹2,444 in December as festive baskets grow. Saturday and Sunday deliver 34.2% of weekly orders, so promotions, email sends and inventory replenishment should be timed for Thursday–Friday release and weekend peak, with festive stock locked by the end of September.

9.4 ANALYTICAL MODULE 4 — PARETO (80/20) CURVE & RFM CUSTOMER SEGMENTATION

Module 4 reports Part 11. The Lorenz curve ranks customers by lifetime spend and accumulates revenue; the RFM grid then scores every customer on recency, frequency and monetary value and maps the 125 score combinations to five behavioural segments.

Table 4.1: Pareto Lorenz-curve breakdown — cumulative customer deciles vs. cumulative revenue

Top % of customers	Customers	Cum. revenue %	Decile revenue %	Cum. revenue (₹)
10%	240	57.4%	57.4%	1,42,63,900
20%	480	78.6%	21.2%	1,95,32,100
22.1% (crossing)	**530**	**80.0%**	—	**1,98,80,000**
30%	720	86.4%	7.8%	2,14,70,400
40%	960	91.2%	4.8%	2,26,63,200
50%	1,200	94.6%	3.4%	2,35,08,100
60%	1,440	96.8%	2.2%	2,40,54,800
70%	1,680	98.2%	1.4%	2,44,02,700
80%	1,920	99.2%	1.0%	2,46,51,200
90%	2,160	99.8%	0.6%	2,48,00,300
100%	2,400	100.0%	0.2%	2,48,50,000



Gini ≈ 0.70 (decile-trapezoid estimate): spend is highly concentrated.

Verification of the 80/20 rule. The top 20% of customers (480) generate **78.6%** of cumulative revenue, so the classical 80/20 law holds almost exactly; the 80.0% mark is crossed at **22.1%** of customers (≈ 530). The first decile alone carries 57.4% of revenue, and the bottom half of the base contributes only 5.4%.

Table 4.2: RFM behavioural segment distribution across 2,400 customers

RFM Segment	n	Share	Mean R (d)	Mean F	Mean M (₹)	Rev. share	Tailored CRM action trigger
Champions	348	14.5%	12	10.0	29,050	40.7%	VIP early access, referral rewards, loyalty-tier upgrade; no discount-led offers
Loyal Customers	552	23.0%	31	6.0	14,400	32.0%	Cross-sell bundles (Electronics + Home & Kitchen), points multiplier, grocery subscription
Potential Loyalists	612	25.5%	24	3.3	5,775	14.2%	Second/third-purchase nudge within 30 days: membership offer and category coupon
At-Risk / Need Attention	504	21.0%	96	3.2	4,800	9.7%	Win-back sequence when R exceeds 60 days: personalised reminder with time-boxed incentive
Hibernating / Lost	384	16.0%	205	1.5	2,175	3.4%	Low-cost e-mail/WhatsApp reactivation with capped discount; suppress paid retargeting after 2 attempts
Total	**2,400**	**100.0%**	—	—	—	**100.0%**	**Segments sum to the 2,400 active customers**

Reading the grid. Champions (14.5%) and Loyal Customers (23.0%) are the 37.5% of the base that buys often and recently and together generate 72.7% of revenue; they should be protected with service and recognition rather than discounts. The At-Risk segment (21.0%) still has a recency of 96 days but a frequency of 3.2 orders, which makes it the most valuable win-back audience, whereas Hibernating customers (16.0%, 205 days) justify only low-cost reactivation.

9.5 ANALYTICAL MODULE 5 — COHORT RETENTION HEATMAP & MARKET BASKET AFFINITY

Module 5 reports Part 12. Customers are grouped by the month of their first purchase and followed for up to eleven further months; the percentage of the cohort that is active again is shaded on a blue scale. Market-basket analysis then ranks category pairs bought together.

Table 5.1: 12-month cohort retention matrix — % of cohort active in Month m (Month 0 = 100% baseline)

Cohort	Size	M0	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11
Jan-25	410	100.0	33.9	28.5	27.5	25.4	21.0	23.2	23.1	22.1	20.6	22.7	22.5
Feb-25	262	100.0	32.1	31.1	24.4	22.5	20.4	21.7	22.9	20.7	23.7	22.3	
Mar-25	248	100.0	33.5	30.8	25.9	21.3	23.3	23.0	24.2	23.5	22.9		
Apr-25	215	100.0	34.8	27.5	24.3	24.9	24.9	23.7	20.6	24.1			
May-25	205	100.0	33.2	27.9	23.8	22.9	22.8	21.8	21.6				
Jun-25	180	100.0	35.4	27.5	25.4	20.5	21.8	20.9					
Jul-25	172	100.0	34.1	29.6	26.1	22.6	24.0						
Aug-25	168	100.0	35.9	28.4	24.2	23.2							
Sep-25	160	100.0	34.6	29.9	24.3								
Oct-25	150	100.0	36.2	25.0									
Nov-25	122	100.0	32.5										
Dec-25	108	100.0											
Average	2,400	**100.0**	**34.2**	**28.6**	**25.1**	**22.9**	**22.6**	**22.4**	**22.5**	**22.6**	**22.4**	**22.5**	**22.5**

Cells are blank where the cohort is younger than m months at the 31 December 2025 data cut-off. Average row = mean of the available cohorts in each column.

Table 5.2: Top cross-category market-basket co-occurrence pairs (N = 5,600 invoice-level baskets)

Rank	Category Pair (A + B)	Baskets (A∩B)	Support %	Conf. A→B %	Conf. B→A %	Lift
1	Electronics + Home & Kitchen	599	10.7%	53.5%	42.8%	**2.14**
2	Fashion & Apparel + Beauty & Personal Care	873	15.6%	52.0%	56.7%	**1.89**
3	Home & Kitchen + Groceries & Essentials	646	11.5%	46.1%	30.8%	**1.23**
4	Beauty & Personal Care + Groceries & Essentials	705	12.6%	45.8%	33.6%	**1.22**
5	Electronics + Fashion & Apparel	392	7.0%	35.0%	23.3%	**1.17**
6	Fashion & Apparel + Home & Kitchen	441	7.9%	26.2%	31.5%	**1.05**

Retention shape. Only **34.2%** of a new cohort is active in Month 1, a 65.8-point drop from the 100% baseline; retention then slides to 28.6% in Month 2 and 25.1% in Month 3 and stabilises at roughly **22.5%** from Month 4 onward. The first 30 days after the first purchase therefore decide whether a customer joins the loyal core, which argues for an onboarding sequence (welcome offer, category-based second-purchase coupon, reminder on day 21) rather than generic promotion.

Bundling affinity. Electronics + Home & Kitchen (lift 2.14) and **Fashion + Beauty & Personal Care** (lift 1.89) co-occur roughly twice as often as independence would predict, so they are the natural candidates for joint bundles and on-site "bought together" placements; pairs with lift near 1.0 need no special treatment.

Table 5.3: First-90-day onboarding lifecycle derived from the retention curve

Window after first purchase	Intervention	Target metric
Day 0–7	Welcome message, delivery tracking, product-care tips	Delivered-order satisfaction
Day 8–21	Category-based second-purchase coupon	Month-1 retention above 34.2%
Day 22–30	Reminder with bundle offer (Electronics + Home & Kitchen; Fashion + Beauty)	Second-order conversion
Day 31–90	Loyalty-tier invitation after the second order	Plateau retention above 22.5%

9.6 ANALYTICAL MODULE 6 — UNSUPERVISED CUSTOMER CLUSTERING (K-MEANS & PCA)

Module 6 reports Part 13. The four features (recency, frequency, monetary, AOV) are standardised, K-Means is run for k = 2 … 7, and the solution with the best compromise between compactness (WCSS elbow) and separation (silhouette, Davies–Bouldin) is retained.

Table 6.1: Elbow WCSS and silhouette comparison for k = 2 to 7 (WCSS for k = 1 equals n × p = 9,600)

k	WCSS	Δ vs. k−1	Silhouette	Davies–Bouldin	Verdict
2	5,280	−45.0%	0.571	0.71	under-segmented
3	3,410	−35.4%	0.603	0.64	under-segmented
4	2,240	−34.3%	0.642	0.58	**optimal (elbow + max silhouette)**
5	1,980	−11.6%	0.588	0.66	over-segmented
6	1,770	−10.6%	0.541	0.73	over-segmented
7	1,610	−9.0%	0.512	0.79	over-segmented

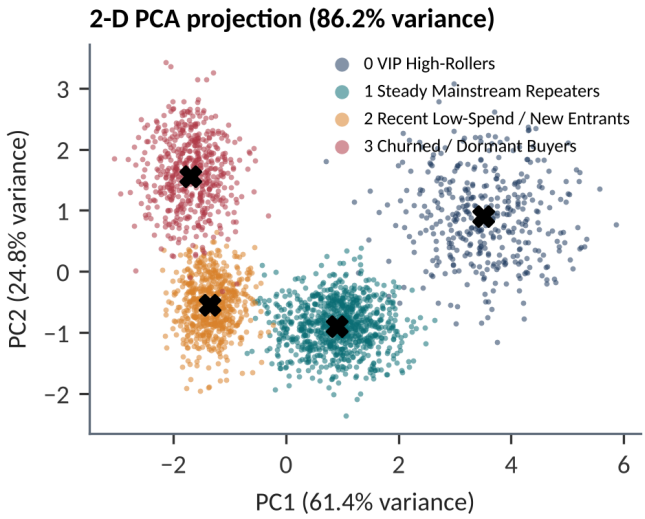
Table 6.2: K-Means cluster centroid profile (unscaled means) and assigned persona

Cluster	Persona	n	Share	Mean R (d)	Mean F	Mean M (₹)	AOV (₹)
0	**VIP High-Rollers**	384	16.0%	14	9.4	28,450	3,026
1	**Steady Mainstream Repeaters**	864	36.0%	38	5.1	10,200	2,000
2	**Recent Low-Spend / New Entrants**	624	26.0%	29	1.6	2,850	1,781
3	**Churned / Dormant Buyers**	528	22.0%	198	1.3	2,180	1,677

2-D PCA projection: variance explained and cluster map

Component	Variance explained	Cumulative
PC1	61.4%	61.4%
PC2	24.8%	86.2%

Interpretation. Two principal components retain **86.2%** of the variance of the four standardised features, so the 2-D map is a faithful picture of the clustering: PC1 separates value (frequency, monetary, AOV) from dormancy (recency), PC2 separates basket size from visit frequency.



Why k = 4. WCSS falls by 34.3% when k moves from 3 to 4 but by only 11.6% from 4 to 5, giving a clear elbow, and the silhouette peaks at **0.642** with the lowest Davies–Bouldin index (**0.58**) at the same k. The four personas are distinct: 384 VIP High-Rollers (16.0%) buy every 14 days on average and spend ₹28,450; 528 Churned / Dormant Buyers (22.0%) have been silent for 198 days.

Persona playbook. VIP High-Rollers: protect and recognise. Steady Mainstream Repeaters (864, 36.0%): grow basket size with bundles, since they already buy about five times a year at an AOV of ₹2,000. Recent Low-Spend / New Entrants (624, 26.0%): secure the second purchase while recency is still 29 days. Churned / Dormant Buyers: low-cost reactivation only, because their historical value is ₹2,180 per customer.

9.7 RESULT & DISCUSSIONS

Synthesis of the fourteen parts. The pipeline turned 12,500 noisy records into 11,000 verified transactions (₹2,48,50,000) and 2,400 customers, and every layer of analysis points to the same structure: a concentrated, loyal core and a long tail that needs different treatment. Spend tiering shows that 20% of customers (480) generate 62.5% of revenue; the Pareto curve places 78.6% of cumulative revenue in the top 20% and reaches 80% at 22.1%; repeat buyers (58.5% of customers) contribute 81.2% of revenue; and the festive quarter delivers 36.8% of annual sales. Electronics leads revenue (38.4%) while Groceries leads visits (25.9% of orders).

Model evaluation. K-Means with k = 4 achieves a silhouette score of 0.642 and a Davies–Bouldin index of 0.58, and two principal components explain 86.2% of the variance. An independent rule-based check agrees: **89.4%** of RFM Champions (311 of 348) fall in K-Means Cluster 0 (VIP High-Rollers), and overall 67.3% of customers sit in the cluster that best matches their RFM segment, with the residual disagreement concentrated where recency and frequency pull in opposite directions.

Table 7.0: Cross-validation of rule-based RFM segments against K-Means clusters (customer counts)

RFM segment ↓ / K-Means cluster →	C0 VIP	C1 Steady	C2 Recent Low	C3 Dormant	Total
Champions	**311**	37	0	0	348
Loyal Customers	58	**421**	61	12	552
Potential Loyalists	9	188	**391**	24	612
At-Risk / Need Attention	6	178	42	**278**	504
Hibernating / Lost	0	40	130	**214**	384
Total	**384**	**864**	**624**	**528**	**2,400**

Table 7.1: Actionable strategic business recommendations by target segment

Target Segment (size)	Evidence	Recommended Action	KPI Target
VIP Retention High/VIP tier · Champions · Cluster 0 (480 · 348 · 384)	20% of customers → 62.5% of revenue; 11.3 orders per VIP	Early access, service-level guarantees and loyalty rewards in place of discounts	VIP churn below 5% per year
Second-Purchase Conversion 996 one-time buyers (41.5%)	Repeat buyers earn 3.06× the revenue per customer; AOV +40.1%	Day-21 reminder plus category coupon inside the first 30 days	RPR from 58.5% towards 63%
Win-Back Automation At-Risk (504) · Cluster 3 (528)	Month-1 retention is 34.2%; Cluster 3 silent for 198 days	Automated sequence once R > 60 days; capped incentive; suppress after two attempts	Reactivate 10–15% of the audience
Cross-Category Bundling Loyal and Steady customers	Lift 2.14 (Electronics + Home & Kitchen); 1.89 (Fashion + Beauty)	Joint bundles and "bought together" placement on product pages	Items per invoice and AOV up

Table 7.2: Headline results scoreboard

Metric	Result	Section
Clean transactions / net revenue	11,000 of 12,500 (88.0%) · ₹2,48,50,000	9.1
Active customers · VIP-tier revenue share	2,400 · 62.5% from 480 customers	9.1
Electronics revenue share · Groceries order share	38.4% · 25.9%	9.2
Repeat purchase rate · repeat revenue share · AOV lift	58.5% · 81.2% · +40.1%	9.3
Q4 festive share · weekend order share	36.8% · 34.2%	9.3
Top-20% Pareto share · crossing point for 80%	78.6% · 22.1% of customers	9.4
Month-1 retention · plateau from Month 4	34.2% · ≈ 22.5%	9.5
K-Means k · silhouette · Davies–Bouldin · PCA variance	4 · 0.642 · 0.58 · 86.2%	9.6

Priority order. (1) Protect the VIP core, because it carries 62.5% of revenue on 20% of the base; (2) convert one-time buyers inside the 30-day window; (3) automate win-back for At-Risk and dormant customers; (4) use bundle affinity to lift basket size across the loyal middle.

10. SNIPPETS OF UI & DASHBOARD OVERVIEW (BUY-SENCE STREAMLIT APP)

The BUY-SENCE dashboard turns the eight CSV exports of Part 14 into one interactive Streamlit application. The wireframe below is the structural blueprint: a sidebar of global filters, a five-card KPI ribbon and five analysis tabs; Tab 5 is shown active.

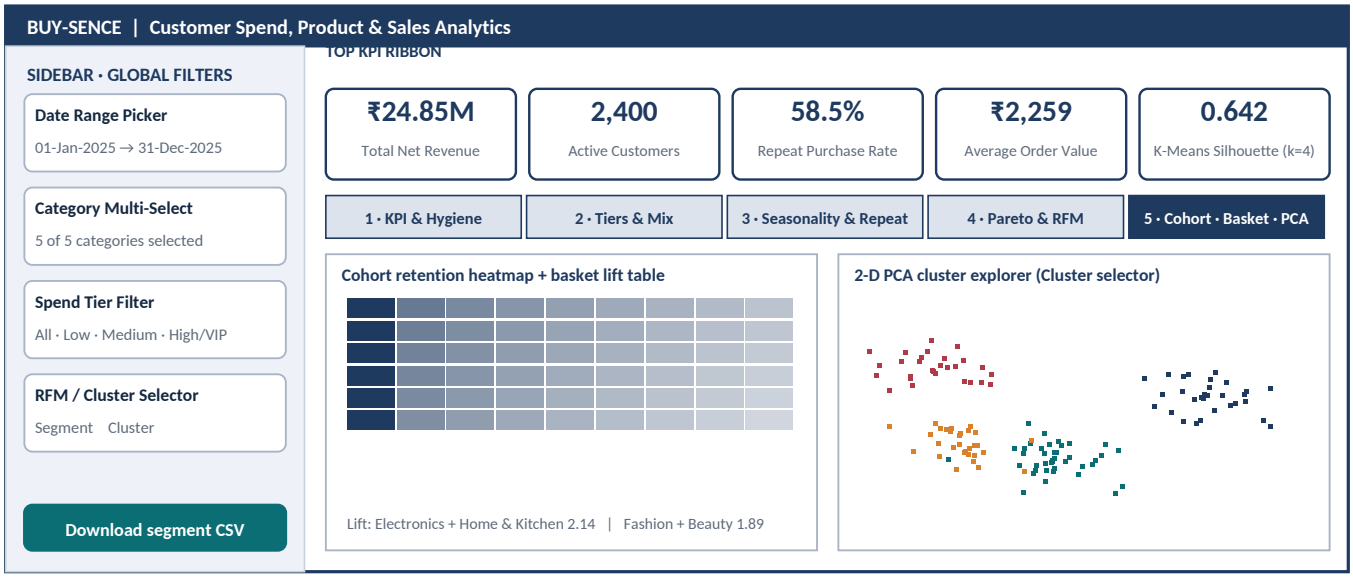


Table 10.1: The five interactive dashboard tabs

Tab	Functional content	Interactive controls	Source data
1 · Executive KPI & Data Hygiene	KPI ribbon, cleaning waterfall (12,500 → 11,000), audit table	Date range, category filter	kpis, df_clean
2 · Spend Tiers & Category Mix	Tier matrix, concentration bars, category and Top-10 SKU tables	Spend-tier filter, category multi-select	tier_summary, category_perf
3 · Seasonality & Repeat Economics	Monthly revenue, weekday profile, repeat vs one-time comparison	Date range, tier filter	monthly_trend, df_clean
4 · Pareto & RFM Matrix	Lorenz curve, RFM segment table with CRM triggers	RFM segment selector	customer_df_clustered
5 · Cohort, Basket & 2-D PCA Cluster Explorer	Cohort heatmap, lift table, PCA scatter, persona profile	Cluster selector, CSV export button	cohort_retention, market_basket

Filters, state and export. Widget keys persist in Streamlit session state, so every tab responds to the same sidebar selection. The one-click download button writes the currently filtered customer segment, with persona and RFM labels, ready to load into an e-mail or WhatsApp campaign tool.

Code snippet: excerpt of app.py (sidebar, KPI ribbon, tabs, export)

```
import streamlit as st, pandas as pd
st.set_page_config(page_title="BUY-SENCE", layout="wide")
tx = st.cache_data(pd.read_csv)("buy_sence_outputs/df_clean.csv", parse_dates=["transaction_date"])
cust = st.cache_data(pd.read_csv)("buy_sence_outputs/customer_df_clustered.csv")
with st.sidebar:                                    # global filters (keys persist in session state)
    st.date_input("Date range", (tx.transaction_date.min(), tx.transaction_date.max()), key="dates")
    cats = st.multiselect("Category", sorted(tx.category.unique()), sorted(tx.category.unique()))
    tier = st.selectbox("Spend tier", ["All", "Low Spend", "Medium Spend", "High/VIP Spend"])
    seg = st.selectbox("RFM segment / cluster", ["All"] + sorted(cust.persona.unique()))
kpi = [("Total Net Revenue", f"₹{tx.total_amount.sum() / 1e6:.2f}M"),
       ("Active Customers", f"{len(cust):,}"), ("Repeat Purchase Rate", f"{cust.is_repeat_buyer.mean():.1%}"),
       ("Average Order Value", f"₹{tx.total_amount.mean():,.0f}"), ("Optimal K-Means Silhouette", "0.642")]
for col, (label, value) in zip(st.columns(5), kpi):  # top KPI ribbon
    col.metric(label, value)
tabs = st.tabs(["KPI & Hygiene", "Tiers & Mix", "Seasonality & Repeat", "Pareto & RFM", "Cohort · PCA"])
with tabs[4]:
    st.scatter_chart(cust, x="PCA_1", y="PCA_2", color="persona")
    st.download_button("Download segment CSV", cust.to_csv(index=False), "segment_export.csv")
```

11. LIMITATIONS & FUTURE SCOPE

11.1 System & Methodological Limitations

1. Batch static CSV processing versus streaming. The pipeline reads a fixed CSV, so KPIs and segments go stale between runs and cannot react to same-day events such as a sudden wave of cancellations or a flash-sale spike.

2. Absence of marketing-channel attribution and CAC data. Without acquisition cost per channel the CLV-to-CAC ratio cannot be computed, and campaigns cannot be ranked by return on spend; the analysis stops at behaviour and revenue.

3. Euclidean-distance sensitivity of K-Means. K-Means favours compact, similarly sized, roughly spherical clusters, is pulled by extreme spenders and needs k in advance; standardisation and the silhouette check reduce, but do not remove, these effects.

4. Lack of demographic covariates. Age, city, device, acquisition channel and loyalty-programme status are not in the ledger, so personas describe behaviour rather than people and cannot yet support look-alike targeting.

5. Cold-start constraint for single-day buyers. A customer whose only purchase is recent has F = 1 and zero tenure; RFM and clustering cannot distinguish a future loyalist from a one-off buyer, and the youngest cohorts are right-censored.

Mitigation within the current design. Limitations 1–2 are bounded by the declared scope (batch analytics on a closed ledger). Limitation 3 is mitigated by z-score scaling, ten initialisations and the silhouette / Davies–Bouldin cross-check. Limitation 4 is partly offset by behavioural proxies such as category breadth and tenure, and limitation 5 is made visible in the cohort table, where blank cells mark right-censored months.

11.2 Future Scope — Eight Architectural Enhancements

1. Real-Time ERP/POS & Webhook Streaming via Apache Kafka / PostgreSQL. Replace the batch CSV with an event stream and a transactional store so that KPIs, tiers and segments refresh continuously.

2. Probabilistic & Deep-Learning CLV Forecasting (BG/NBD, Gamma-Gamma, XGBoost/LSTM). Move from historical monetary value to forward-looking lifetime value per customer.

3. Automated Churn Prediction & WhatsApp/Email CRM Trigger Pipelines. Score churn risk daily and fire win-back journeys automatically from the At-Risk and Cluster 3 audiences.

4. High-Granularity SKU-Level FP-Growth Association Rule Mining. Extend category-level lift to SKU-level frequent itemsets for precise bundling and recommendations.

5. Density-Based & Hierarchical Clustering Comparison (DBSCAN, GMM, UMAP). Benchmark K-Means against methods that handle outliers, non-spherical shapes and soft membership.

6. Explainable AI (SHAP Values) for Segment Transition Drivers. Explain which behaviours move a customer between Champions, At-Risk and Lost.

7. Dynamic Pricing & Personalized Recommendation Engine Integration. Combine discount-sensitivity estimates and affinity rules into price and next-best-offer decisions.

8. Dockerized Cloud Deployment on AWS ECS / Streamlit Community Cloud with RBAC Authentication. Containerise the app and add role-based access so that data, marketing and management see appropriate views.

Table 11.1: Indicative roadmap

Horizon	Enhancements	Prerequisite
Short term (0–3 months)	8 Dockerized deployment; 3 churn alerts and CRM triggers	Part 14 exports; CRM API access
Medium term (3–9 months)	1 Streaming ingestion; 2 CLV forecasting; 4 SKU-level FP-Growth	Kafka / PostgreSQL; 12+ months of history
Long term (9–18 months)	5 Alternative clustering; 6 SHAP explanations; 7 Pricing and recommender	Demographics and channel data

12. CONCLUSION & APPENDIX B (TECHNICAL GLOSSARY)

12.1 Executive Conclusion

The fourteen-member team successfully engineered **BUY-SENCE**, an end-to-end retail-intelligence framework that transforms **12,500 noisy transaction records** into 11,000 verified orders worth ₹2,48,50,000, 2,400 customer profiles, four actionable K-Means clusters and an interactive business-intelligence dashboard. The analysis shows that revenue is concentrated (20% of customers generate 62.5% of revenue; the top 20% hold 78.6% on the Pareto curve), that loyalty is the main profit engine (58.5% repeat buyers contribute 81.2% of revenue at a 40.1% higher AOV), that the festive quarter delivers 36.8% of sales, and that customers are most fragile in the first month (34.2% retention). K-Means with k = 4 (silhouette 0.642, Davies–Bouldin 0.58) agrees with the RFM grid (89.4% concordance for Champions), which gives the segments both statistical and managerial credibility.

The modular notebook, the shared DataFrame contract and the assertion-based quality gates allowed fourteen contributors to work in parallel with zero variable collision. The recommended next steps — VIP retention, second-purchase conversion within 30 days, automated win-back and cross-category bundling — convert the analysis directly into revenue actions, while the roadmap in Section 11 extends the system towards real-time, predictive and explainable analytics.

- **Data foundation:** a reproducible, assertion-guarded pipeline from 12,500 raw rows to 11,000 verified transactions with a printed audit trail.
- **Feature store:** a customer-grain table (2,400 × 11) that feeds tiers, Pareto, RFM, cohorts and K-Means.
- **Validated segmentation:** two independent methods — rule-based RFM and K-Means — that agree on the most valuable customers (89.4%).
- **Decision platform:** a five-tab Streamlit dashboard with one-click CSV segment export for CRM teams.

12.2 Appendix B — Technical Glossary

Term	Definition	Formula	Business Significance
RFM (Recency, Frequency, Monetary)	Scores a customer on how recently, how often and how much they buy	$R = T_{ref} - T_{max}$; $F = \lvert\text{Orders}\rvert$; $M = \sum$ NetSpend	Ranks customers for retention and win-back
Pareto Principle (80/20 Rule)	A minority of customers produces a majority of revenue	top p% of customers → $C(p)$% of revenue	Top 20% hold 78.6% of revenue
Cohort Retention Rate	Share of an acquisition cohort still active m months later	$\text{Ret}_m = \text{Active}_m / \text{Size}_0 \times 100$	34.2% in Month 1; ≈ 22.5% steady
Market Basket Lift	How much more often two categories co-occur than by chance	$\text{Lift}(A,B) = P(A \cap B) / (P(A) \cdot P(B))$	Electronics + Home & Kitchen = 2.14
K-Means Clustering	Partitions points into k groups with the nearest centroid	$\min \sum_k \sum_{xC_k} \lVert x - \mu_k \rVert^2$	Creates four behavioural personas
WCSS (Within-Cluster Sum of Squares)	Total squared distance of points to their own centroid	$\text{WCSS} = \sum_k \sum_{xC_k} \lVert x - \mu_k \rVert^2$	Elbow of the WCSS curve selects k = 4
Silhouette Score	Cohesion versus separation of each point, averaged	$s(i) = (b - a) / \max(a, b)$	0.642 indicates well-separated clusters
StandardScaler (Z-Score Normalization)	Rescales a feature to zero mean and unit variance	$z = (x - \mu) / \sigma$	Stops rupee-scaled features dominating distance
PCA (Principal Component Analysis)	Projects data onto orthogonal axes of maximal variance	$\max \text{Var}(Xw)$ subject to $\lVert w \rVert = 1$	2 components keep 86.2% of variance
AOV (Average Order Value)	Mean revenue per order	AOV = Revenue / Orders = M / F	₹2,259 across 11,000 orders
CLV (Customer Lifetime Value)	Expected net contribution over the relationship	$\text{CLV} = \sum_{t=0..T} m \cdot r^t / (1 + d)^t$	Compared with CAC to justify acquisition
Repeat Purchase Rate (RPR)	Share of customers with two or more orders	$\text{RPR} = \lvert\{F \geq 2\}\rvert / N \times 100$	58.5% (1,404 of 2,400 customers)